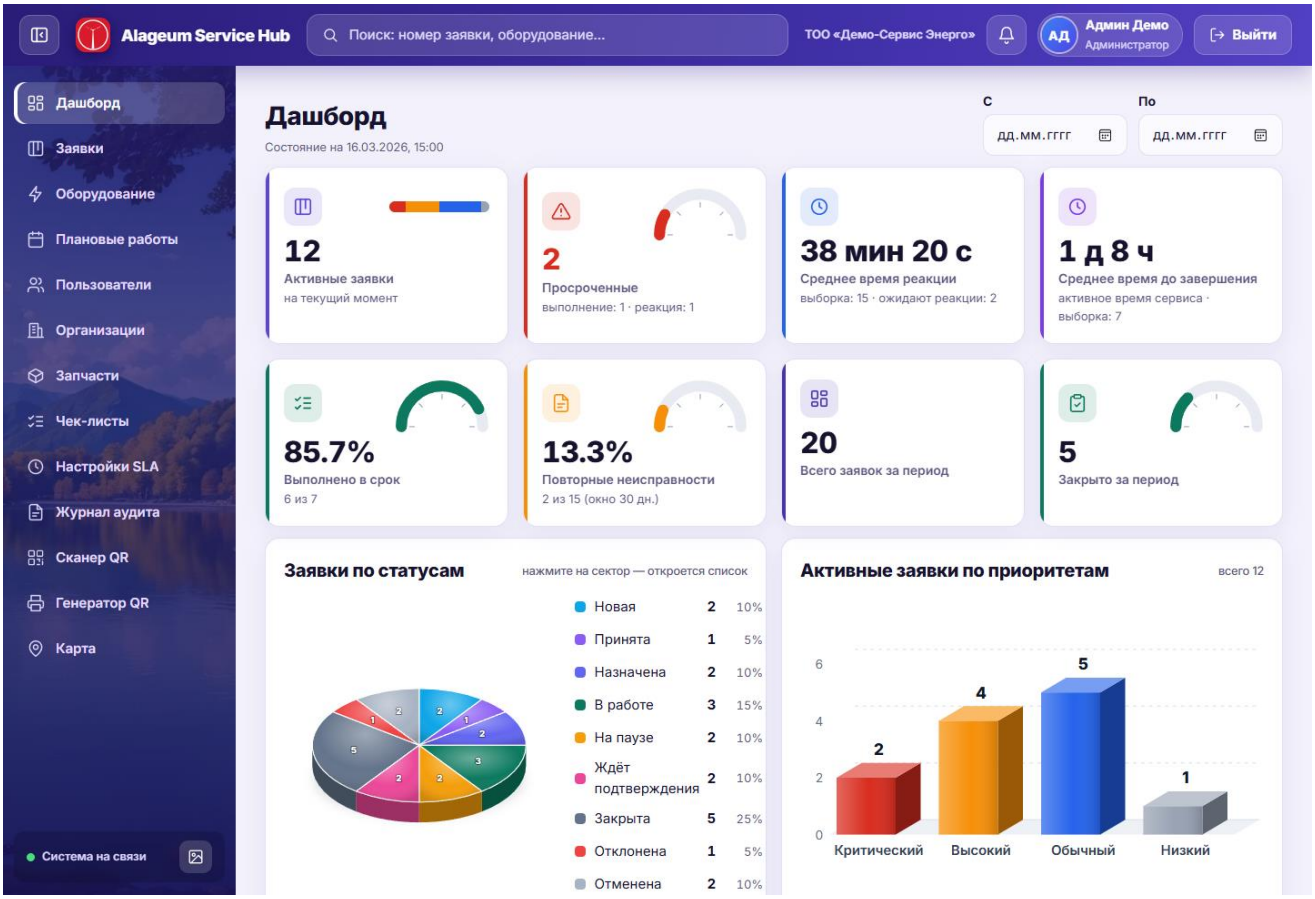
Alageum
Service Hub

Презентационное и техническое описание программы

Учёт электротехнического оборудования
и управление сервисным обслуживанием



Функции и интерфейс • Процесс обслуживания • Архитектура и эксплуатация

Демонстрационная версия MVP

20 октября 2026 года

Содержание

Разделы 1–25 знакомят с продуктом и рабочими сценариями. Разделы 26–34 описывают устройство, сопровождение и приёмку системы.

01 Назначение и возможности платформы 3
02 Роли и границы ответственности 4
03 Сквозной процесс обслуживания 5
04 Вход и рабочая навигация 6
05 Дашборд и управленческий обзор 7
06 Доска заявок 8
07 Список календарь и шкала SLA 9
08 Регистрация новой заявки 10
09 Принятие и назначение исполнителя 11
10 Реестр оборудования 12
11 Паспорт и история оборудования 13
12 Карта объектов 14
13 QR доступ к оборудованию 15
14 Генератор этикеток и документов 16
15 Плановое техническое обслуживание 17
16 Рабочее место инженера 18
17 Отчёт и проверка результата 19
18 Запчасти и использованные материалы 20
19 Журнал заявки и коммуникация 21
20 Акт и подтверждение клиента 22
21 Уведомления и контроль внимания 23
22 Пользователи и административный доступ 24
23 Организации площадки и справочники 25
24 Шаблоны чек листов и настройки SLA 26
25 Мобильный сценарий 27
26 Архитектура приложения 28
27 Модель данных 29
28 Контракт API 30
29 Защита доступа и файлов 31
30 Расчёт SLA и аналитики 32
31 Обновления и фоновые задачи 33
32 Развёртывание и конфигурация 34
33 Резервное копирование и сопровождение 35
34 Приёмка и дальнейшее развитие 36

01 Назначение и возможности платформы

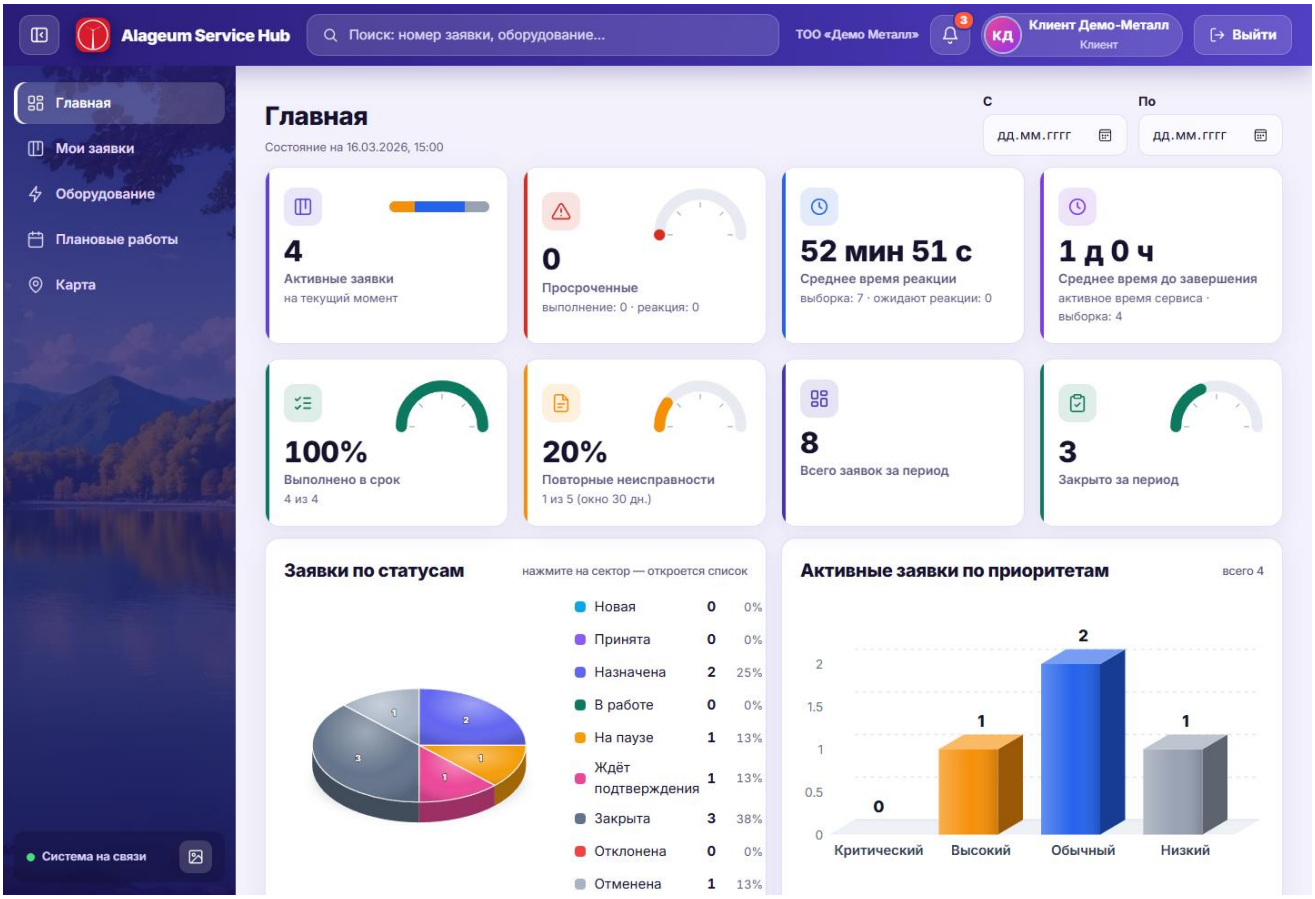


Рисунок 1. Главная страница клиента с показателями и доступными объектами

Alageum Service Hub объединяет учёт электротехнического оборудования и работу сервисной службы. В одной системе связаны организация заказчика, площадка, паспорт оборудования, обращение, исполнитель, отчёт и акт. Эта связь позволяет проследить обслуживание конкретного объекта от регистрации неисправности до подтверждения результата.

Платформа рассчитана на пять ролей: клиента, диспетчера, инженера, руководителя и администратора. Клиент сообщает о проблеме и принимает выполненную работу. Диспетчер управляет очередью и назначениями. Инженер фиксирует диагностику, работы и материалы. Руководитель анализирует сроки и загрузку, а администратор поддерживает пользователей и справочники.

Текущая версия представляет собой демонстрационный MVP студенческой команды Университета имени М. Әуезова. Пилот с Alageum Group предлагается как возможное направление; подтверждённое внедрение или партнёрство не заявляется. Демонстрационные организации, сотрудники и заявки являются учебными данными.

Основа описания: README.md; frontend/src/App.tsx

02 Роли и границы ответственности

Доступ определяется одновременно ролью и областью видимости данных. Наличие ссылки на страницу не даёт права изменить объект. Сервер повторно проверяет полномочия при чтении, загрузке файлов и выполнении действий.

Клиент видит оборудование и заявки своей организации. Инженер получает доступ к назначенным ему заявкам и истории завершённых назначений, а также к связанному оборудованию. Администратор, диспетчер и руководитель имеют общую сервисную область просмотра, но разные права изменения.

Роль	Основные задачи	Граница полномочий
Клиент	Создать обращение, наблюдать ход работ, подтвердить результат или вернуть на доработку	Только своя организация
Диспетчер	Принять заявку, уточнить приоритет, назначить и переназначить инженера, управлять паузой	Оперативное управление сервисом
Инженер	Начать работу, заполнить отчёт и чек-лист, добавить фото и материалы, завершить работу	Действия исполнителя по своему назначению
Руководитель	Просмотреть показатели, заявки, оборудование, планы и загрузку	Аналитика и контроль
Администратор	Управлять пользователями, организациями, шаблонами, SLA и аудитом	Настройка системы; без подмены диспетчера в процессе заявки

Практический пример: администратор может видеть новую заявку, но принятие и назначение в текущей модели выполняет диспетчер. Такой подход разделяет обслуживание системы и ответственность за сервисные решения.

При обращении к объекту вне области видимости API возвращает 404. Если объект доступен для просмотра, но действие запрещено ролью, используется 403. Это различие важно при разборе обращений пользователей о доступе.

Основа описания: backend/app/visibility.py; serializers.py; routers/auth.py

03 Сквозной процесс обслуживания

Основной маршрут заявки проходит через регистрацию, принятие, назначение, выполнение, ожидание подтверждения и закрытие. Промежуточные состояния нужны для управления ответственностью и расчётом сроков, а события сохраняют последовательность решений.

Номер вида SR-2026-0017 служит понятным идентификатором обращения. Внутренние связи построены на UUID. Заявка всегда относится к конкретному оборудованию и площадке, поэтому её история дополняет историю обслуживания объекта.

Этап	Кто действует	Результат
Новая	Клиент или диспетчер	Обращение зарегистрировано; запущены часы SLA
Принята	Диспетчер	Зафиксирована первая реакция и подтверждён приоритет
Назначена	Диспетчер	Выбран инженер; при необходимости задано время
В работе	Назначенный инженер	Начато выполнение и создан черновик отчёта
На паузе	Инженер или диспетчер	Указаны причина, следующий шаг и срок возобновления
Ждёт подтверждения	Инженер завершил работу	Проверен отчёт; сформирован акт
Закрыта	Клиент	Результат принят
Возврат на доработку	Клиент	Работа продолжается с фиксацией замечаний

Отклонение и отмена являются отдельными конечными состояниями. Доступность каждого перехода зависит от роли, текущего статуса и назначения. Интерфейс получает перечень allowed_actions от сервера.

Возврат клиента не стирает предыдущий акт. Система поддерживает версии отчётов и актов, сохраняя связь между первоначальным результатом и доработкой.

Основа описания: backend/app/workflow.py; serializers.py; acts.py

04 Вход и рабочая навигация

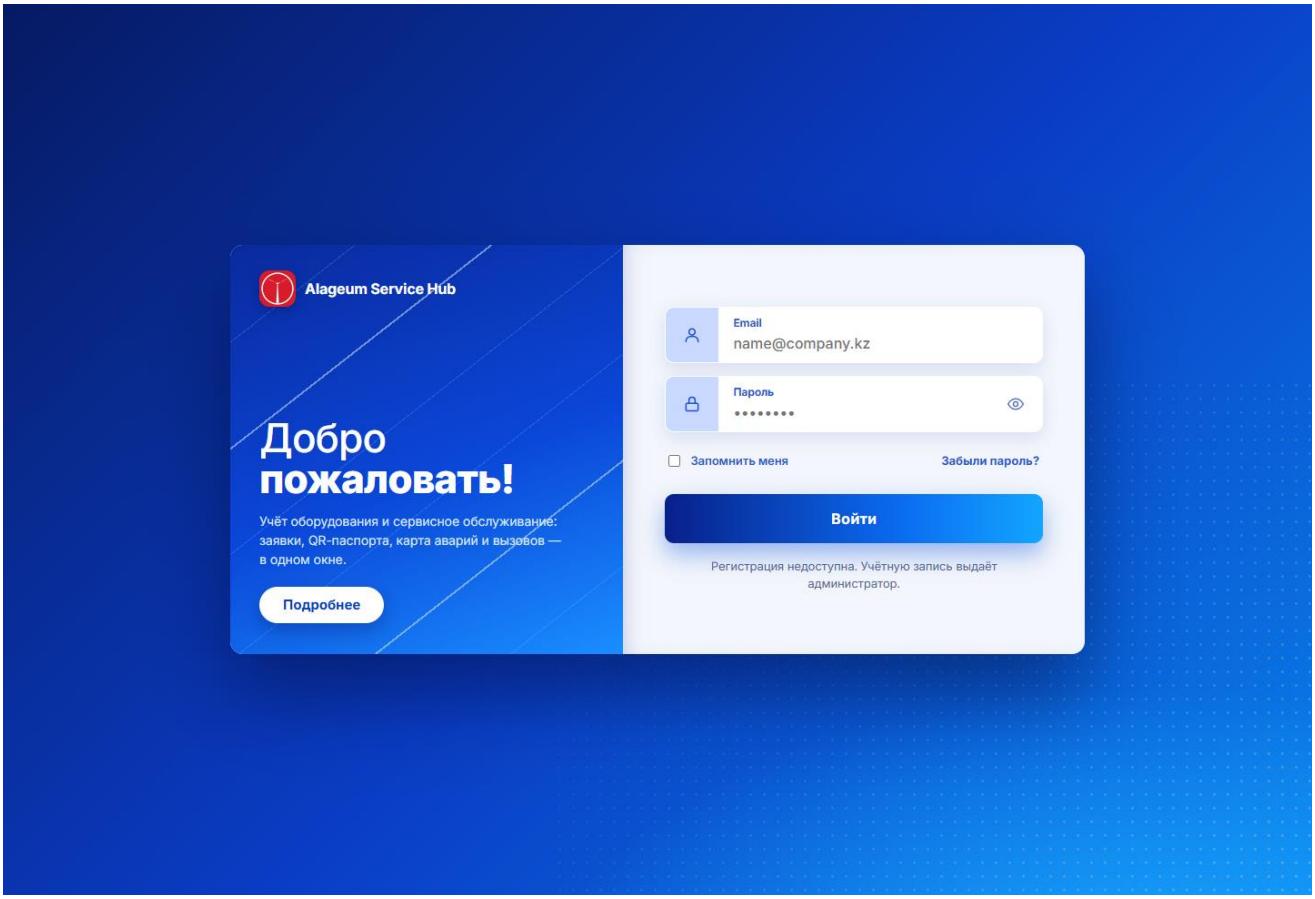


Рисунок 2. Страница входа в Alageum Service Hub

Пользователь входит по email и паролю. После авторизации система открывает рабочее пространство, соответствующее роли: для инженера это задания, для диспетчера — доска заявок. Если администратор включил обязательную смену пароля, сначала открывается соответствующая форма.

На компьютере верхняя панель содержит поиск, уведомления и профиль. Боковое меню показывает разделы текущей роли и может сворачиваться до значков. На телефоне основные переходы перенесены в нижнюю панель, а дополнительные возможности доступны через меню «Ещё».

По умолчанию сеанс завершается после 10 минут бездействия; активность продлевает его. Абсолютный предел составляет 480 минут и задаётся настройкой. Перед завершением интерфейс предлагает продолжить работу. Время сеанса считается по реальным часам, независимо от фиксированного времени демонстрации.

Основа описания: frontend/src/App.tsx; auth/AuthContext.tsx; backend/app/config.py

05 Дашборд и управленческий обзор

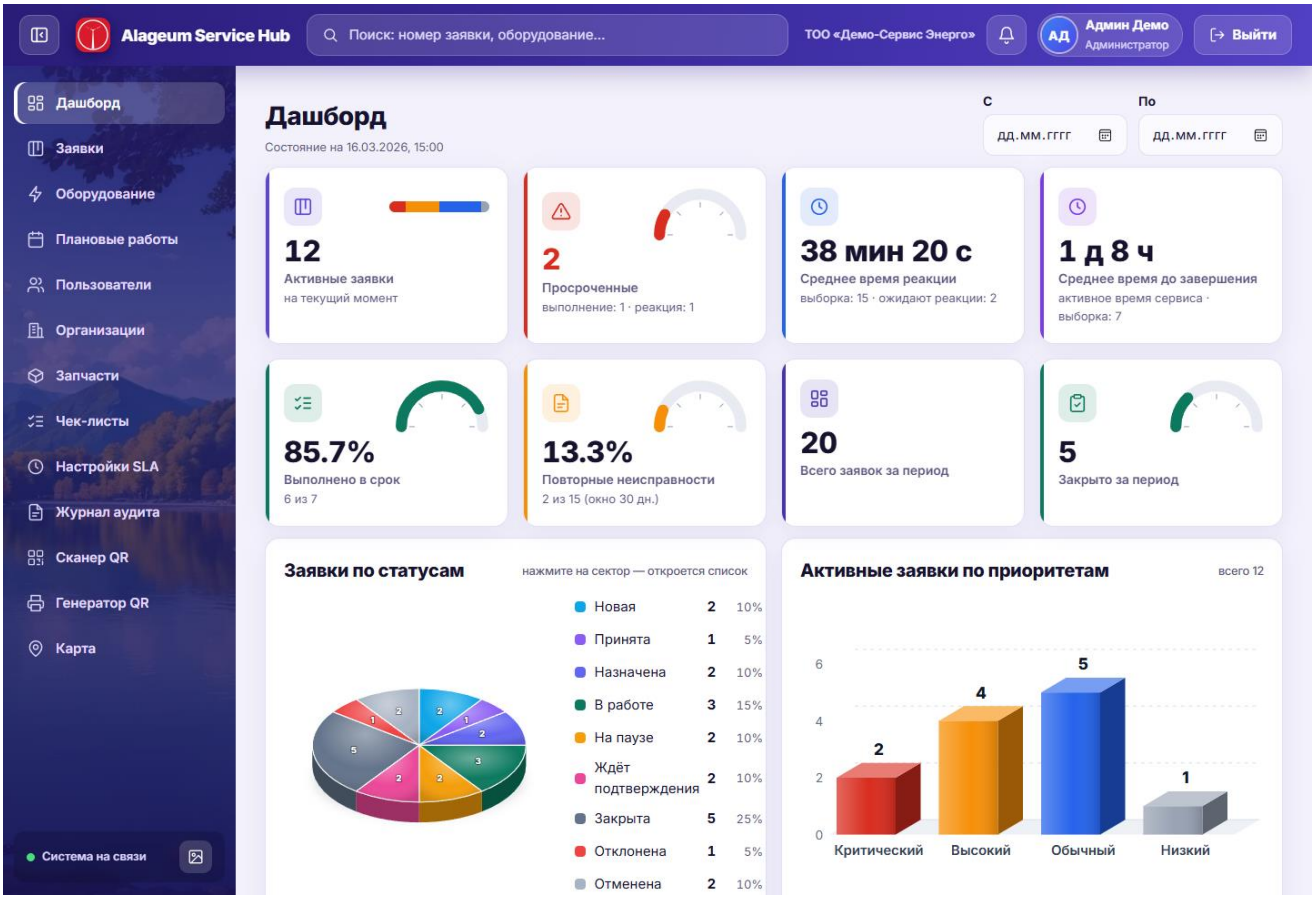


Рисунок 3. Показатели демонстрационного стенда на 16 марта 2026 года

Дашборд помогает оценить очередь обращений и соблюдение сроков. Карточки показывают активные и просроченные заявки, среднее время реакции и выполнения, долю работ в срок, повторные неисправности и количество обращений за период. Диаграммы раскрывают распределение по статусам и приоритетам.

В зафиксированном демонстрационном наборе на экране видны 12 активных заявок, 2 просроченные и 85,7 процента выполнений в срок. Это иллюстрация расчётов на учебных данных, а не показатели деятельности компании. Дата подготовки документа — 20 октября 2026 года; дата на экранах определяется отдельными часами demo.

Нажатие на показатель или сектор диаграммы позволяет перейти к связанным заявкам. При анализе важно различать текущую очередь и выборку за период: активные заявки считаются по состоянию системы, а ряд средних и долей — по обращениям, созданным в выбранном диапазоне.

Основа описания: frontend/src/pages/Dashboard.tsx; backend/app/metrics.py

06 Доска заявок

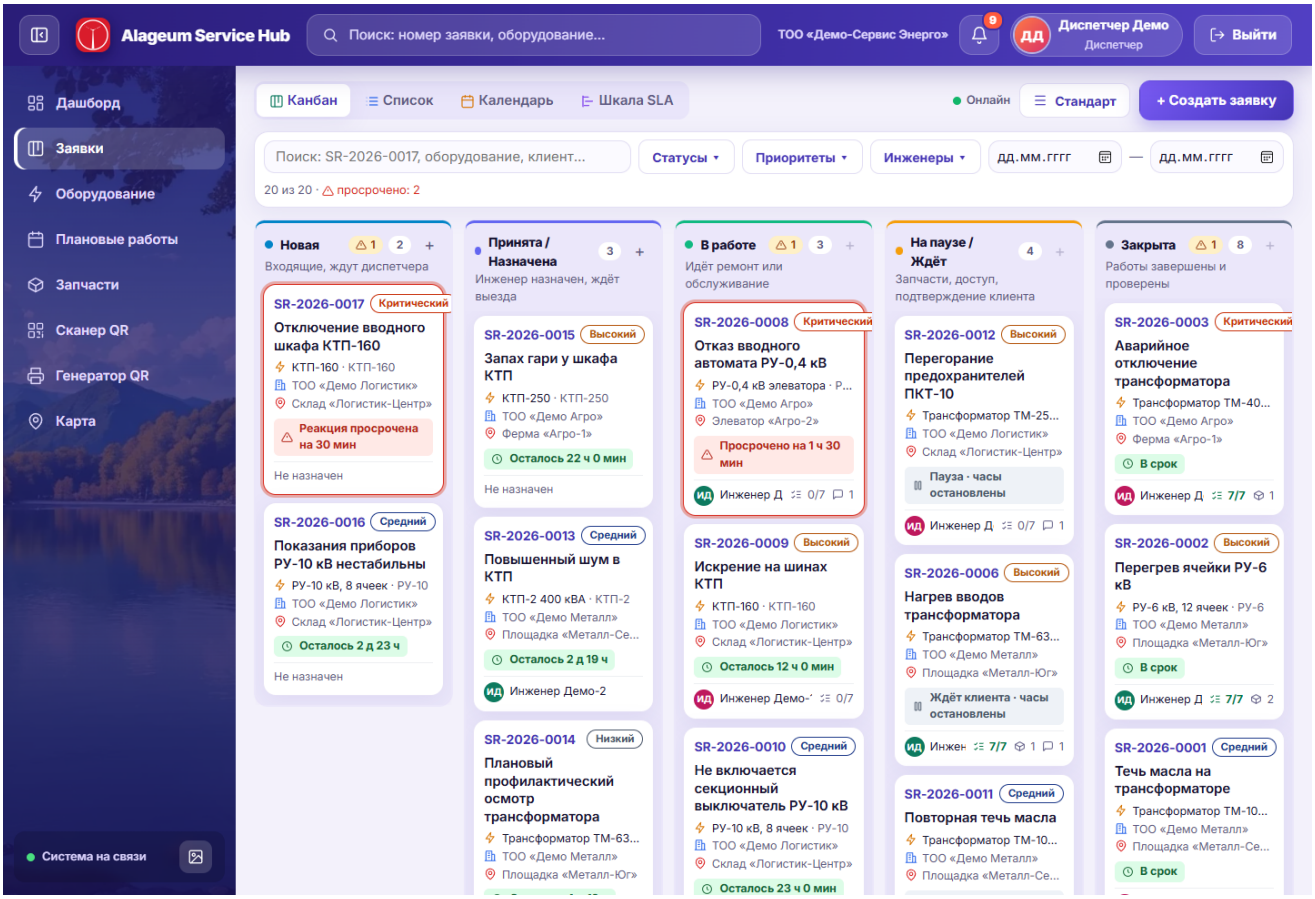


Рисунок 4. Канбан с очередью обращений и индикаторами SLA

Канбан группирует обращения в пять рабочих колонок: «Новая», «Принята / Назначена», «В работе», «На паузе / Ждёт» и «Закрыта». Последняя объединяет также отклонённые и отменённые обращения; подробный статус остаётся виден в карточке и списке.

Карточка содержит номер, приоритет, тему, оборудование, клиента, площадку и исполнителя. Индикатор SLA помогает заметить просрочку или приближение срока. Дополнительные счётчики показывают заполнение чек-листа, материалы и комментарии. Фильтры позволяют сузить очередь по статусам, приоритетам, инженерам, датам и поисковому запросу.

Перетаскивание карточки проверяется по правилам процесса. Если переход требует обязательных данных, открывается диалог; недопустимый перенос не меняет заявку. Настройки представления, фильтров и плотности сохраняются в браузере отдельно для каждого пользователя.

Основа описания: frontend/src/pages/Workspace.tsx; frontend/src/board/

07 Список календарь и шкала SLA

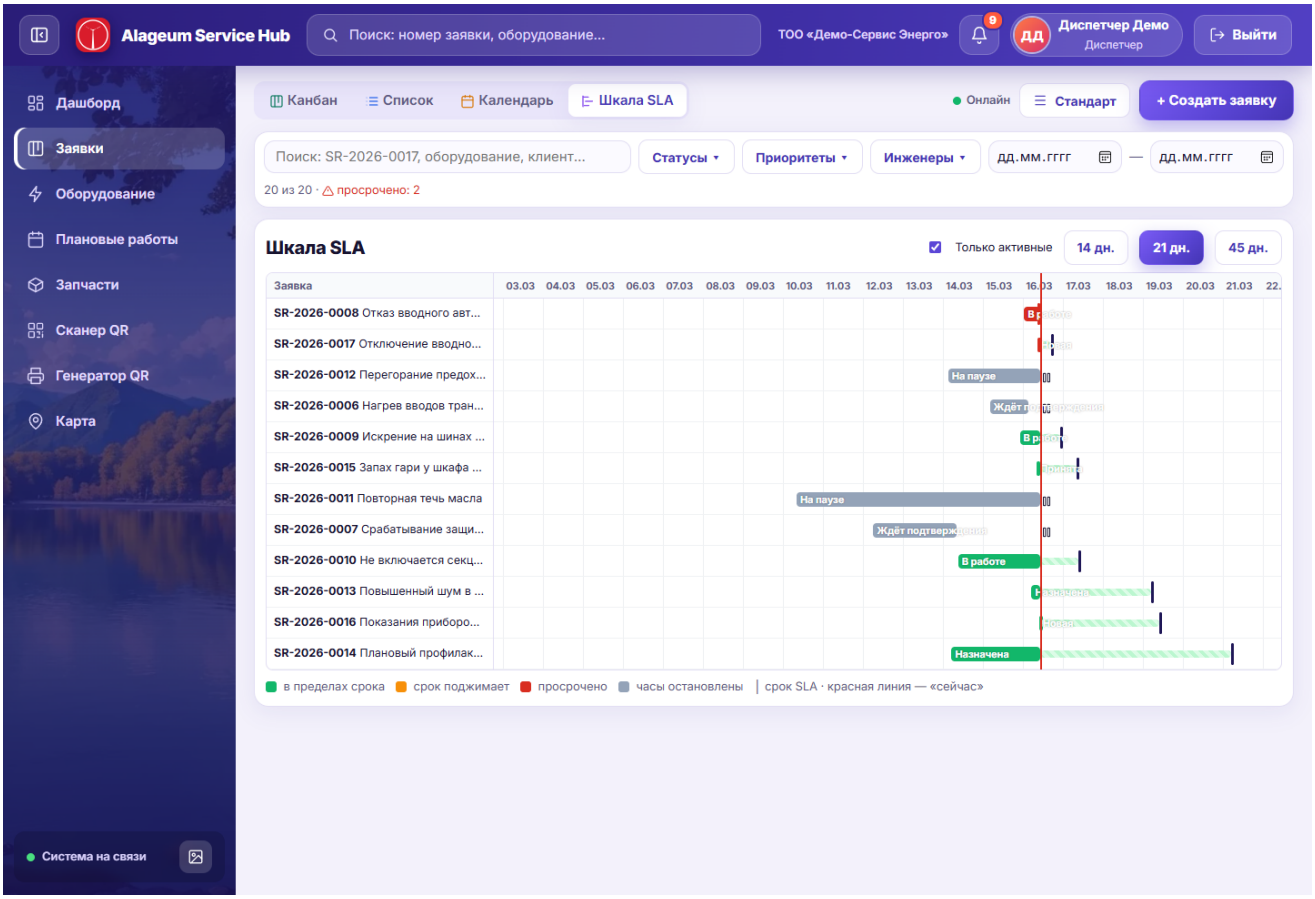


Рисунок 5. Шкала SLA как альтернативное представление той же очереди

Доска имеет четыре режима: канбан, список, календарь и шкалу SLA. Смена режима не создаёт отдельный набор данных. Пользователь выбирает представление под задачу: распределение потока, сравнение строк, просмотр дат или оценку временных интервалов.

В списке удобно сопоставлять номера, статусы и исполнителей. Календарь помогает увидеть размещение работ по датам. Шкала SLA показывает временной контекст обращений и использует часы сервера. Это особенно важно на demo-стенде, где бизнес-время зафиксировано.

Фильтры доски применяются к загруженному набору заявок. Текущая реализация загружает страницы до установленного в интерфейсе предела, поэтому при росте объёма потребуется пересмотреть стратегию загрузки. API списка отдельно поддерживает серверные фильтры и пагинацию.

Основа описания: frontend/src/board/Views.tsx; pages/Workspace.tsx; backend/app/routers/tickets.py

08 Регистрация новой заявки

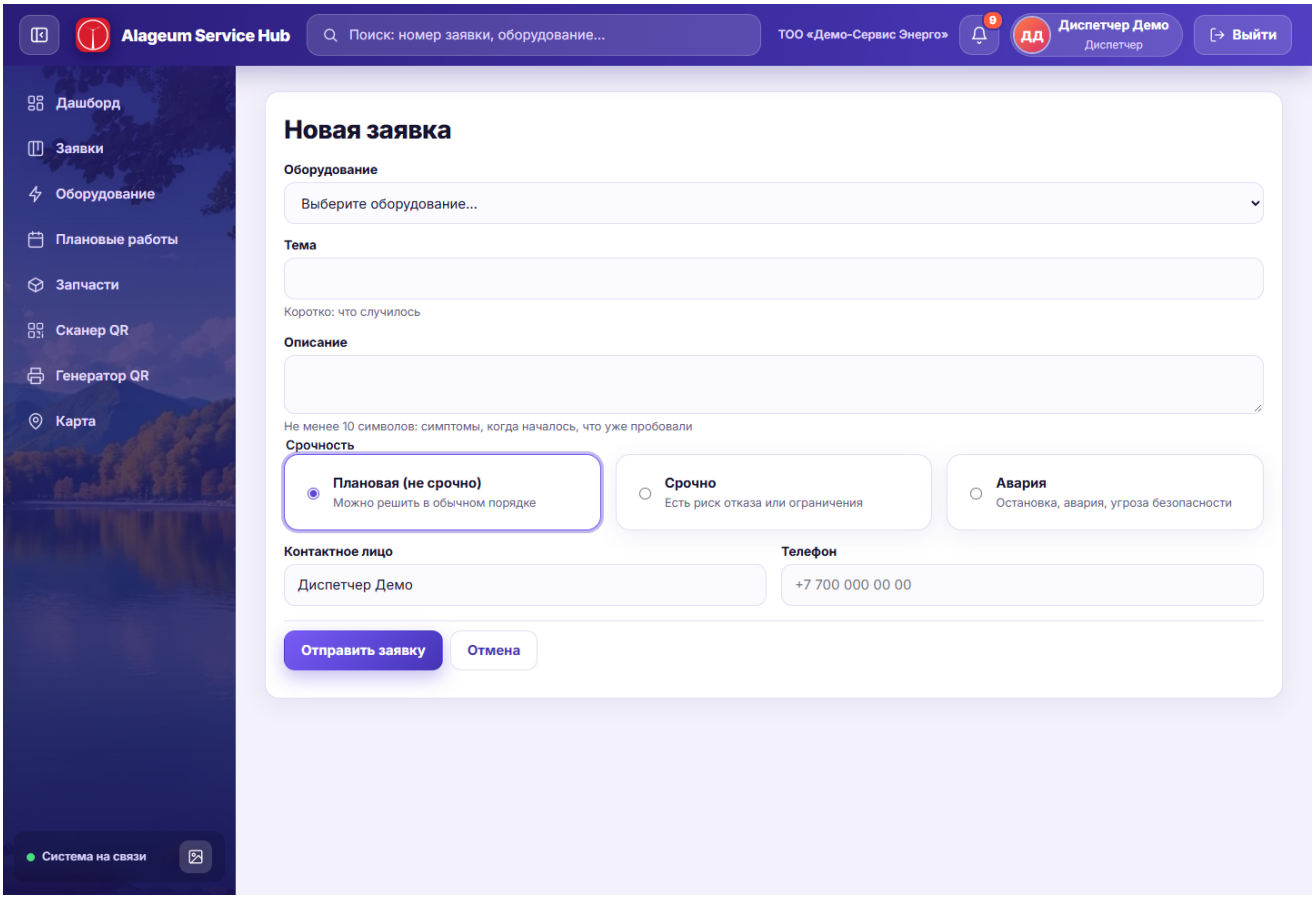


Рисунок 6. Форма обращения с выбором оборудования и описанием проблемы

Заявку создаёт клиент либо диспетчер. Сначала выбирается оборудование, затем указываются тема, описание, срочность и контактные данные. Связь с оборудованием определяет организацию и площадку обращения. Архивные объекты не допускаются к созданию новых заявок.

Срочность преобразуется в исходный приоритет: плановая — обычный, срочная — высокий, аварийная — критический. При принятии диспетчер подтверждает приоритет. Для клиентского обращения источник фиксируется как web, для регистрации диспетчером — phone.

Если по оборудованию уже существует активная корректирующая заявка, система выдаёт предупреждение и номер найденного обращения. Это помогает проверить возможный дубль, но не заменяет решение диспетчера. В описании полезно указать наблюдаемый симптом, место проявления и обстоятельства обнаружения.

Основа описания: backend/app/workflow.py create_ticket; frontend/src/pages/Tickets.tsx

09 Принятие и назначение исполнителя

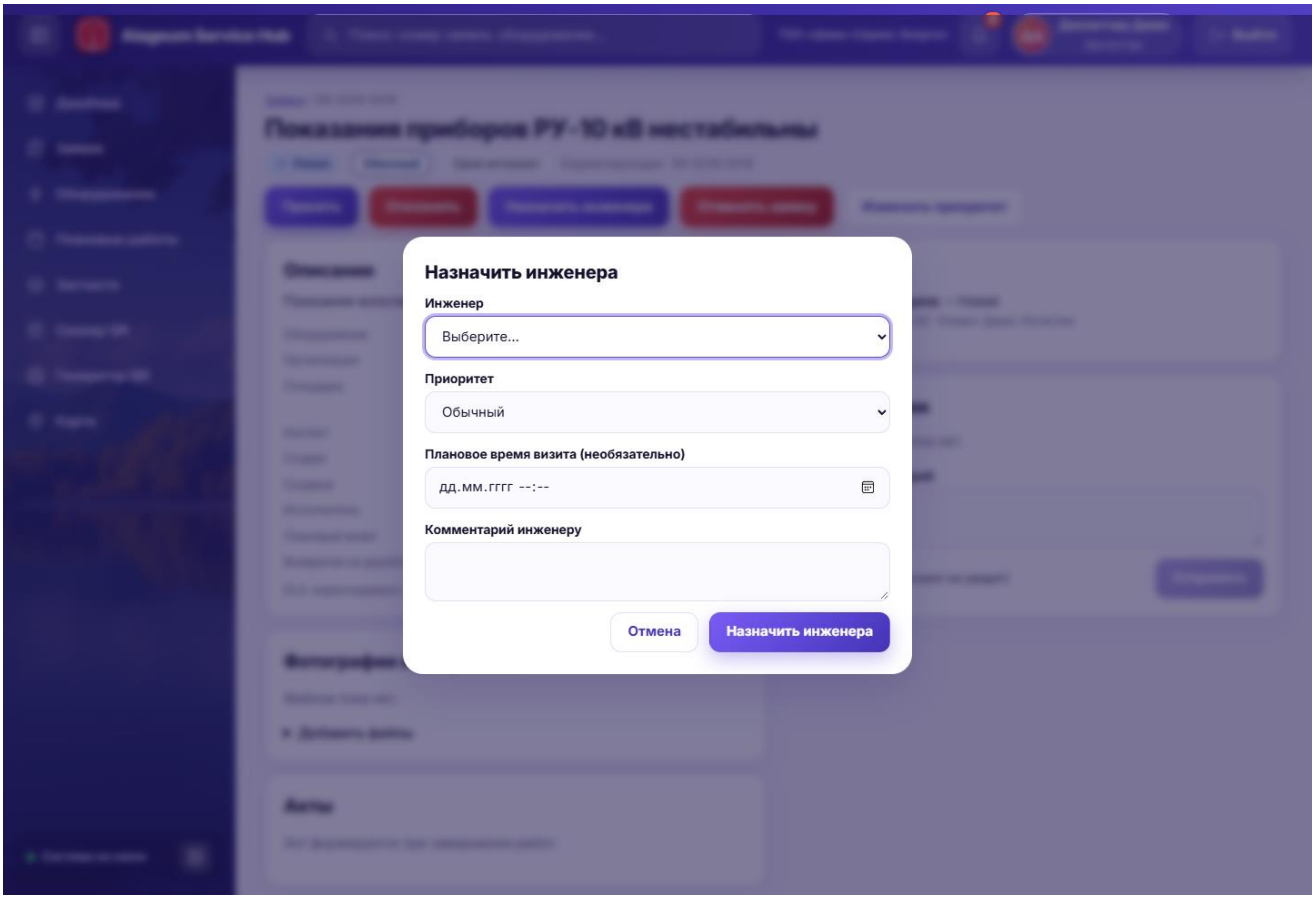


Рисунок 7. Диалог назначения инженера на заявку

Диспетчер проверяет обращение, определяет приоритет и выбирает инженера. Назначение возможно сразу из новой заявки: в этом случае система также выполняет принятие и фиксирует первую реакцию. В форме можно указать плановое время и пояснение.

При выборе исполнителя учитывается его текущая загрузка. В расчёт входят назначенные, выполняемые и приостановленные заявки. Если достигнута заданная вместимость инженера, система предупреждает о перегрузке. Предупреждение помогает принять решение, но само по себе не блокирует назначение.

Переназначение сохраняет историю: предыдущее назначение закрывается, создаётся новое, фиксируются участники и причина. Инженер получает уведомление о назначении, клиент — информацию об исполнителе. Управлять назначениями могут только роли, предусмотренные процессом.

Основа описания: backend/app/workflow.py _do_assign and _do_reassign

10 Реестр оборудования

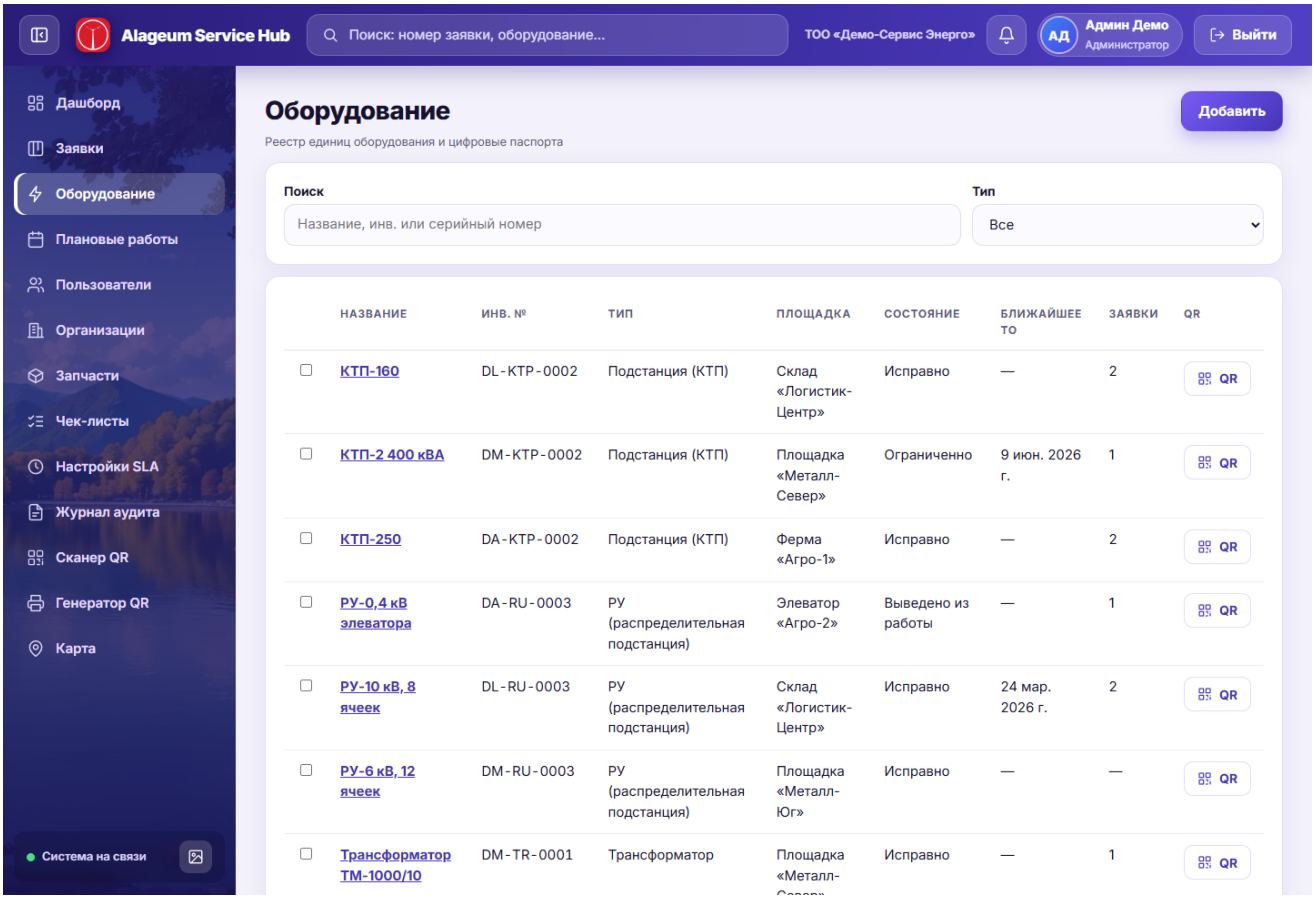


Рисунок 8. Список оборудования с техническими состояниями и переходами к паспортам

Реестр содержит трансформаторы, комплектные подстанции и распределительные устройства. Поиск помогает найти объект по названию, инвентарному или серийному номеру. Для каждой записи сохраняются владелец, площадка, тип, производитель, модель и технические характеристики.

Инвентарный номер уникален внутри организации. Сервер контролирует согласованность организации и площадки, поэтому оборудование нельзя связать с чужой площадкой через обычное изменение идентификатора. Состояния объекта включают исправное, ухудшенное и выведенное из эксплуатации.

Администратор и диспетчер поддерживают карточки оборудования. Архивирование используется для прекращения активной работы с записью при сохранении истории. Клиент видит только собственные объекты, инженер — оборудование в своей области назначений.

Основа описания: backend/app/models.py Equipment; routers/equipment.py; visibility.py

11 Паспорт и история оборудования

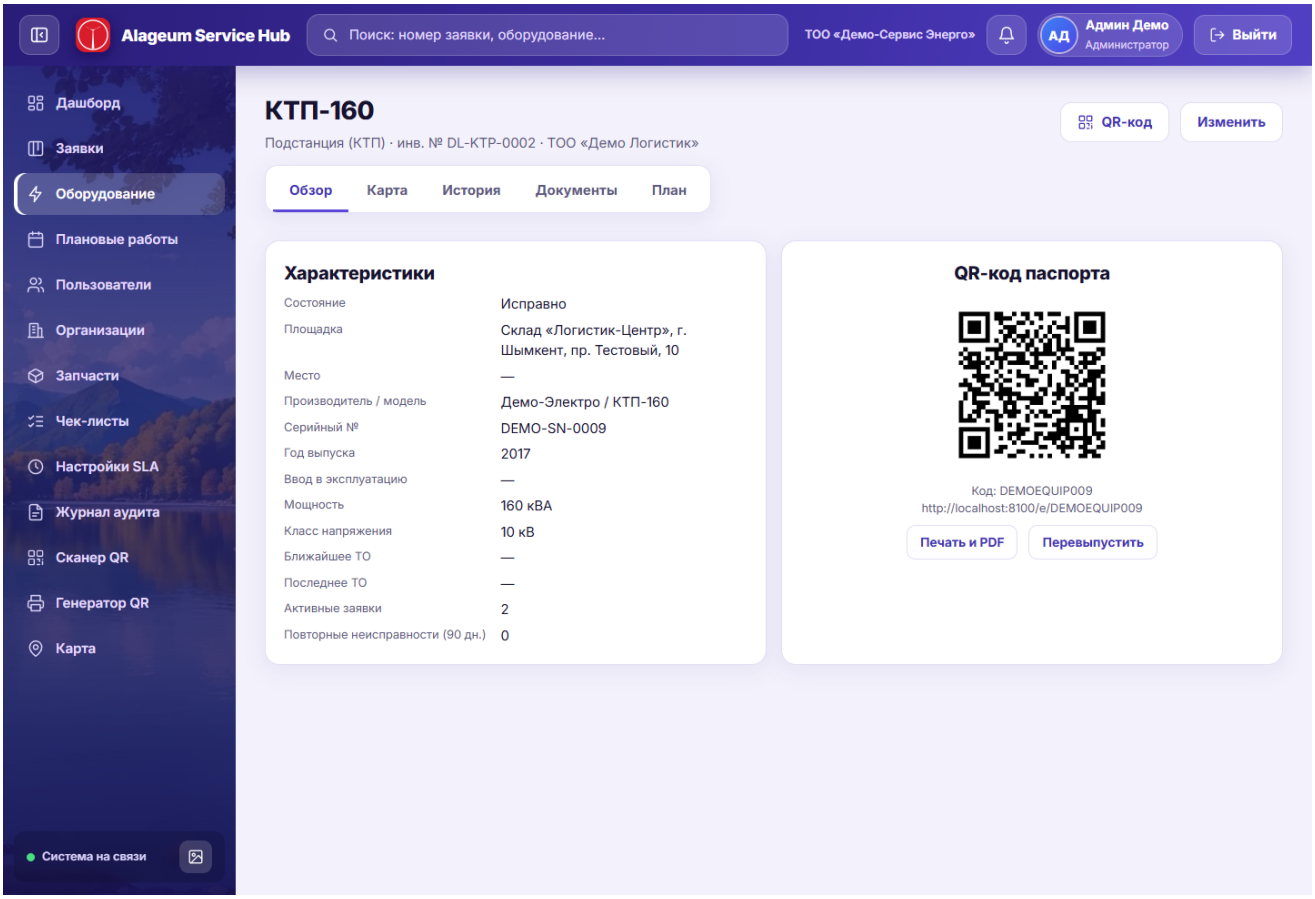


Рисунок 9. Карточка оборудования с вкладками обзора карты истории документов и плана

Паспорт объединяет идентификационные и технические сведения: наименование, модель, серийный номер, год изготовления, дату ввода, класс напряжения, мощность и состояние. Дополнительные паспортные параметры хранятся в структурированном поле и могут различаться по типам оборудования.

Вкладка «История» собирает связанные обращения и позволяет восстановить последовательность обслуживания. Вкладка «Документы» предназначена для файлов оборудования. В разделе «План» показаны регламентные работы, интервалы и следующие даты. Карта помогает связать паспорт с фактическим размещением.

Из карточки можно перейти к QR-коду и сформировать паспорт в PDF. При подготовке к эксплуатации важно согласовать правила заполнения паспортов: единообразные названия, инвентарные номера, единицы измерения и ответственного за актуальность сведений.

Основа описания: frontend/src/pages/Equipment.tsx; backend/app/generators/documents.py

12 Карта объектов

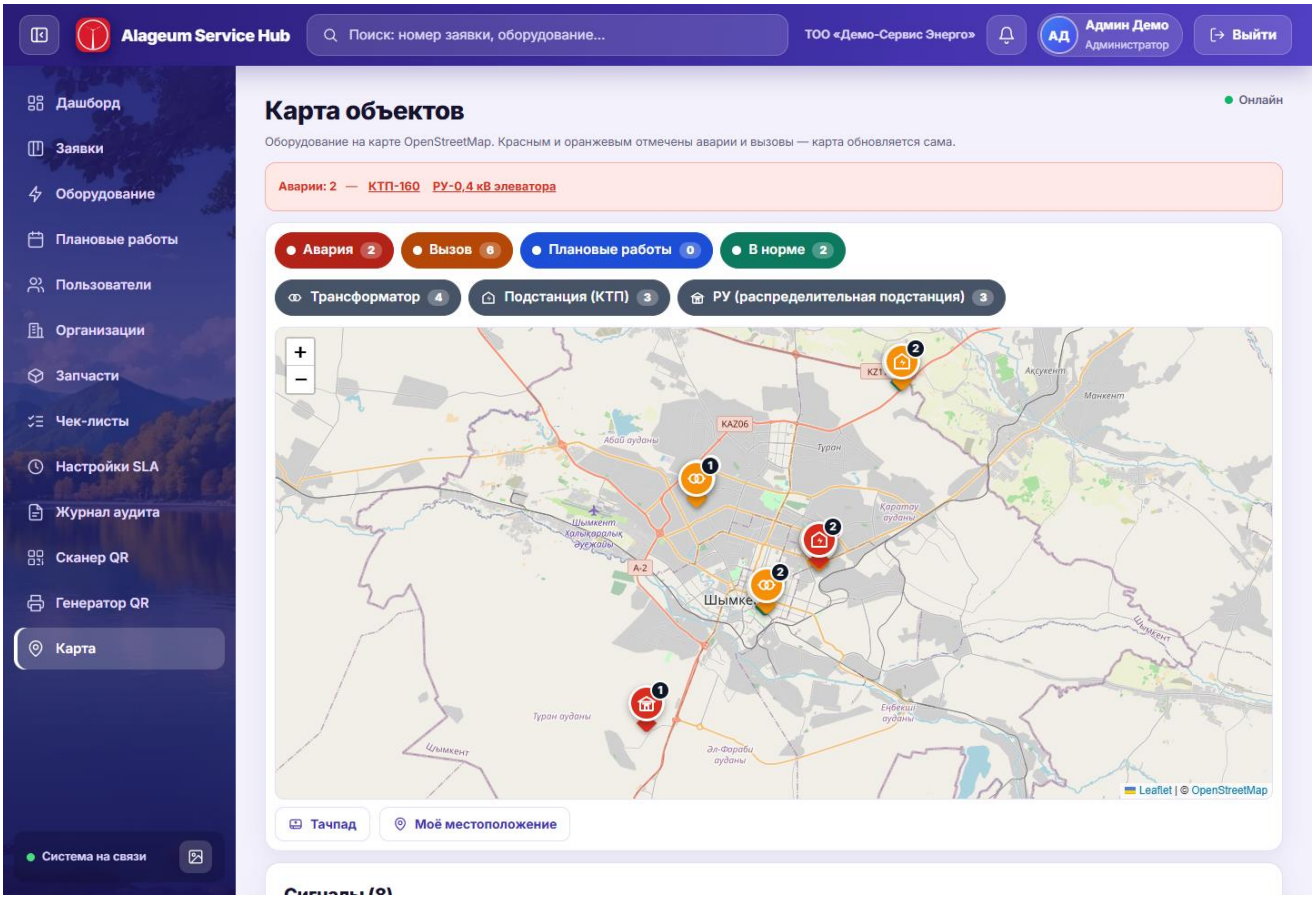


Рисунок 10. Карта оборудования с фильтрами и сигналами состояния

Карта связывает сервисную ситуацию с расположением оборудования. Объект использует собственные координаты, а при их отсутствии — координаты площадки. В демонстрации площадки расположены в районе Шымкента. Значки различают типы электротехнических объектов.

Цветовой сигнал показывает аварийную ситуацию, активный вызов, плановые работы либо нормальное состояние. Сигналы выводятся из состояния оборудования и связанных заявок. Фильтры помогают выделить нужные типы и ситуации, а список объектов — перейти к конкретной метке и паспорту.

Редактор положения поддерживает ввод координат, выбор на карте и перетаскивание метки. Геолокация пользователя используется для ориентирования и доступна при разрешении браузера в защищённом контексте. Подложка OpenStreetMap загружается через Nginx с кэшем; для новых тайлов серверу требуется интернет.

Основа описания: frontend/src/pages/MapPage.tsx; components/LeafletMap.tsx; backend/app/routers/equipment.py

13 QR доступ к оборудованию

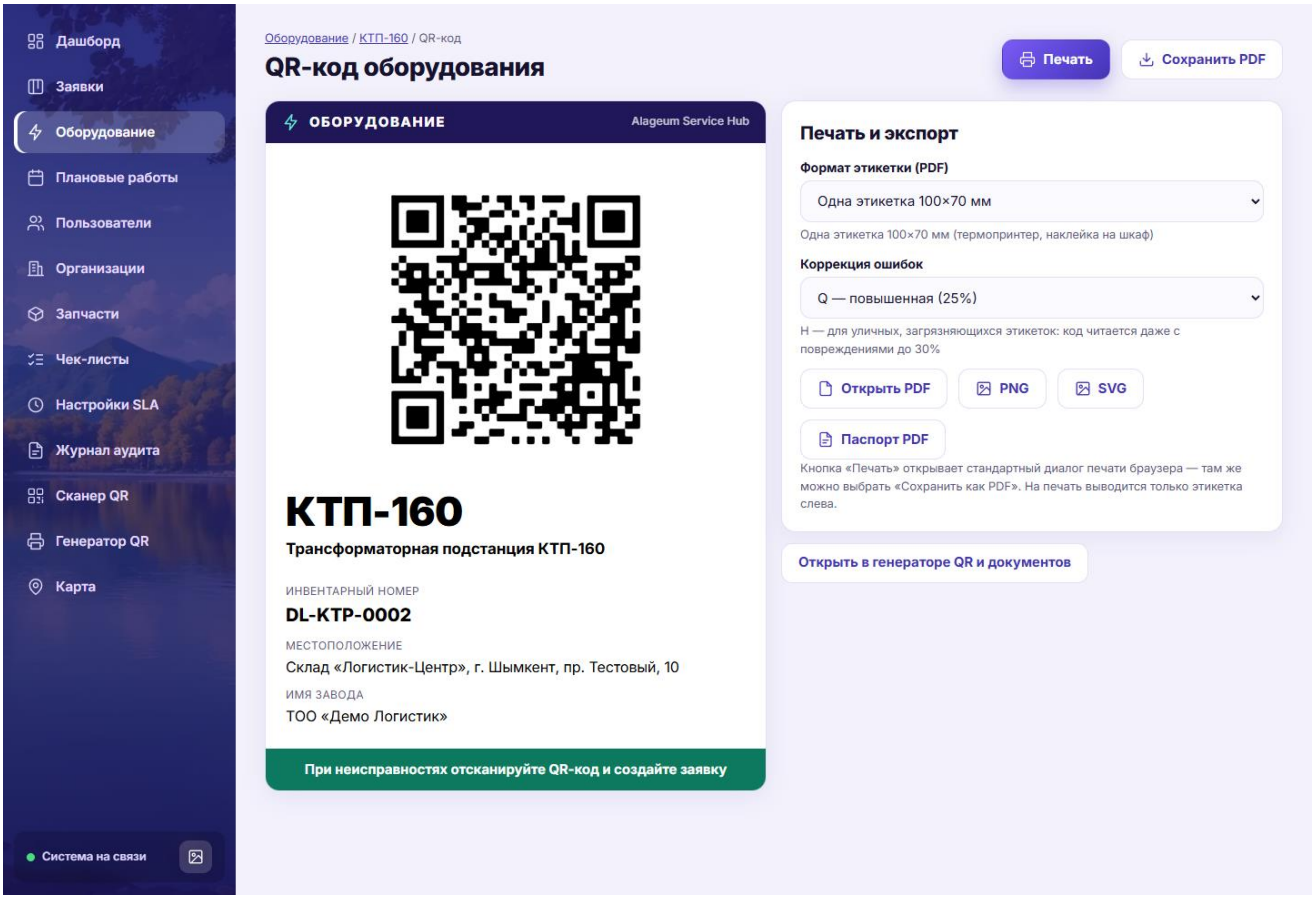


Рисунок 11. QR-код объекта и варианты печати и экспорта

QR-код содержит ссылку на карточку оборудования. После сканирования пользователь проходит авторизацию, если сеанс отсутствует, и получает доступ в пределах своей роли. Код ускоряет поиск объекта на площадке, но не отменяет проверку прав на сервере.

Для отдельного объекта доступны печать, PDF-этикетка, PNG и SVG. Предусмотрены форматы 100 × 70 мм, 50 × 30 мм и лист A4 на восемь этикеток. На этикетке размещаются идентификаторы и сведения, помогающие человеку проверить, что код относится к нужному оборудованию.

Адрес ссылки определяется настройкой APP_PUBLIC_URL. В режиме auto он зависит от адреса, по которому открыт сайт. Для постоянных наклеек перед печатью следует установить устойчивый адрес сервиса: уже наклеенный QR-код не изменится при смене адреса стенда.

Основа описания: frontend/src/pages/QrPage.tsx; backend/app/publicurl.py; generators/qr.py

14 Генератор этикеток и документов

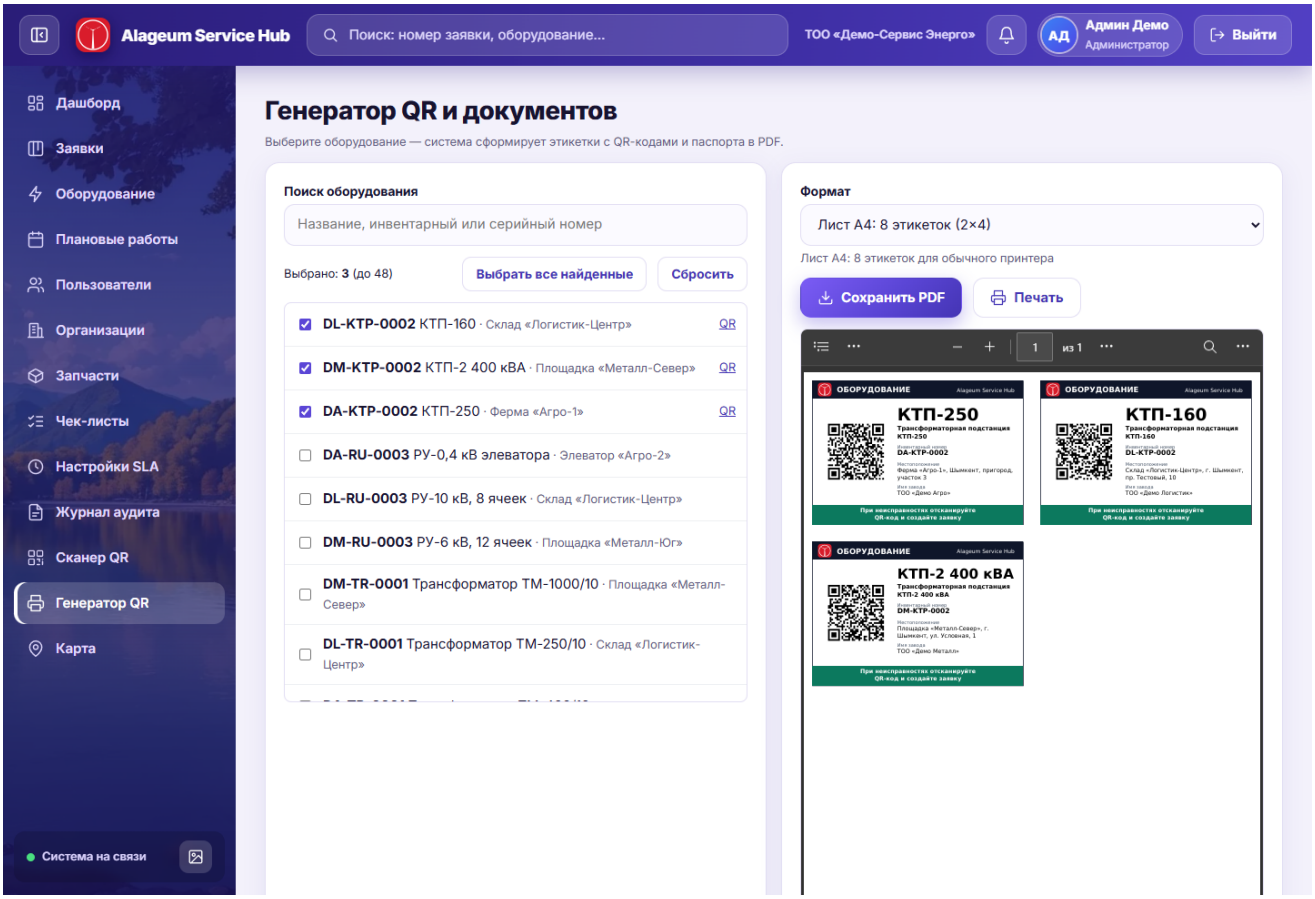


Рисунок 12. Пакетный выбор оборудования и предпросмотр печатного документа

Генератор позволяет выбрать несколько объектов и подготовить общий PDF с QR-этикетками. Поиск сокращает список, а счётчик показывает количество выбранных записей. В текущем интерфейсе пакет ограничен 48 объектами.

После выбора формата справа появляется предпросмотр. Пользователь может скачать PDF или открыть печать. Для одного выбранного объекта также доступен паспорт. Перед массовой печатью полезно проверить на одном экземпляре читаемость кода, адрес перехода и соответствие размера используемому носителю.

Встроенный сканер дополняет этот сценарий: приложение поддерживает распознавание QR через доступные браузеру средства и обработку изображения. Камера зависит от HTTPS и разрешений устройства. При ограничениях камеры остаются поиск объекта и переход по известной ссылке.

Основа описания: frontend/src/pages/QrPage.tsx; pages/Scan.tsx; frontend/src/lib/

15 Плановое техническое обслуживание

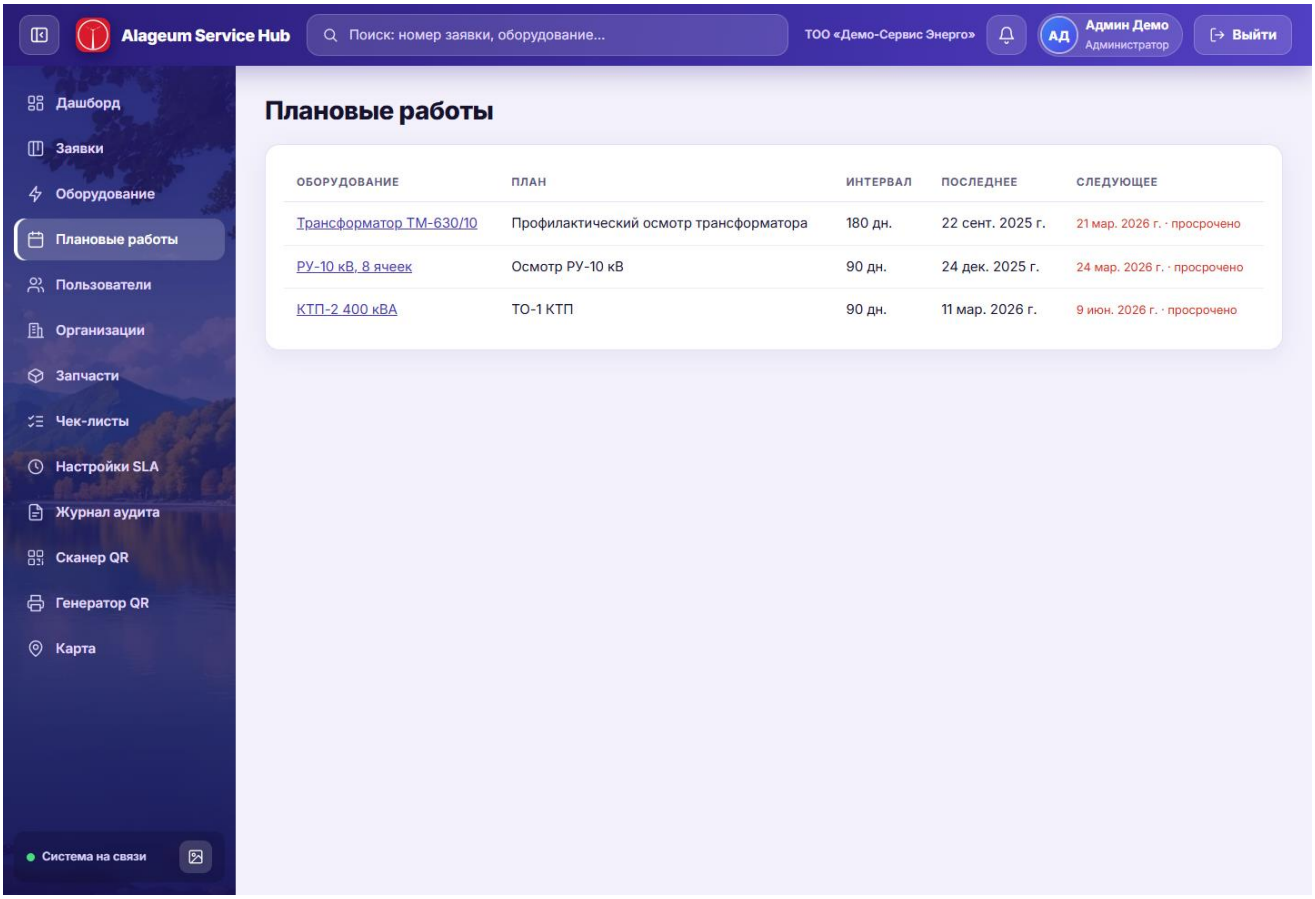


Рисунок 13. Перечень планов и предстоящих работ

План технического обслуживания связывает оборудование с регулярной работой. Он содержит название, описание, интервал в днях, дату следующего выполнения, приоритет и срок предварительного создания заявки. План можно активировать или приостановить.

Фоновый обработчик проверяет активные планы и создаёт заявки при наступлении окна подготовки. Расчёт даты учитывает часовой пояс площадки. Связка плана и плановой даты уникальна, что предотвращает повторное создание одной и той же плановой заявки при следующих проходах worker.

После создания плановая работа проходит общий процесс назначения, выполнения и подтверждения. Это сохраняет единые правила отчёта и истории. Перед применением в реальном сервисе интервалы и чек-листы необходимо согласовать с ответственными специалистами по конкретному оборудованию.

Основа описания: backend/app/jobs.py generate_planned; models.py MaintenancePlan; workflow.py

16 Рабочее место инженера

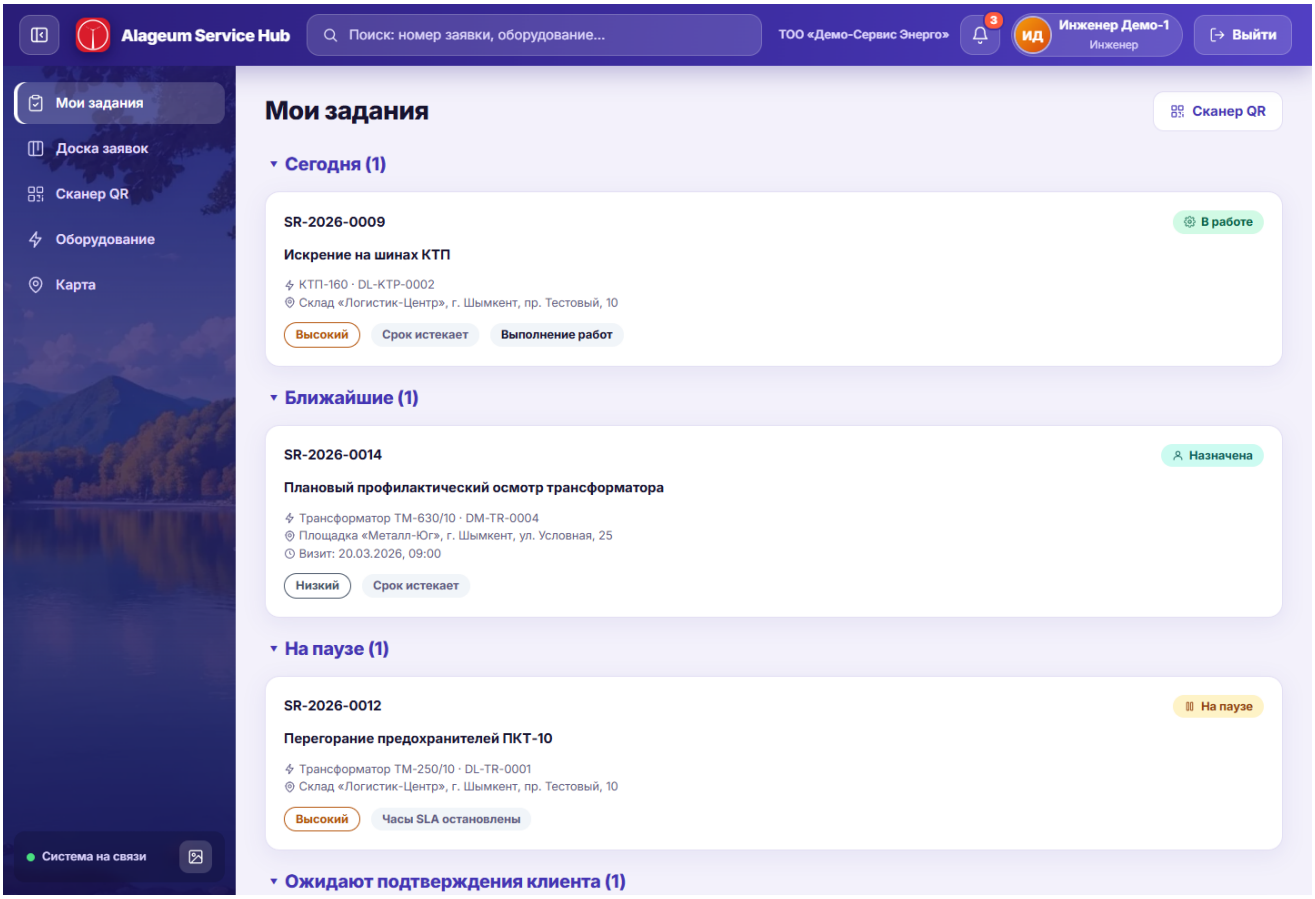


Рисунок 14. Мои задания с приоритетами статусами и сроками

Раздел «Мои задания» собирает обращения, доступные инженеру. Он позволяет определить ближайшие работы, проверить приоритет и открыть карточку оборудования. Права на действия зависят от текущего назначения: просмотр истории не означает возможность изменить чужую текущую работу.

После команды начала система переводит заявку в работу и подготавливает черновик отчёта. Инженер фиксирует диагностику, выполненные операции, результат, чек-лист и материалы. Сведения сохраняются в контексте конкретной заявки, чтобы позже войти в акт.

При необходимости паузы нужно указать причину, пояснение, следующий шаг и ожидаемое время возобновления. Пауза останавливает накопление времени выполнения SLA, но оставляет обращение активным. По истечении ожидаемого срока система может сформировать напоминание.

Основа описания: frontend/src/pages/Engineer.tsx; backend/app/workflow.py; jobs.py

17 Отчёт и проверка результата

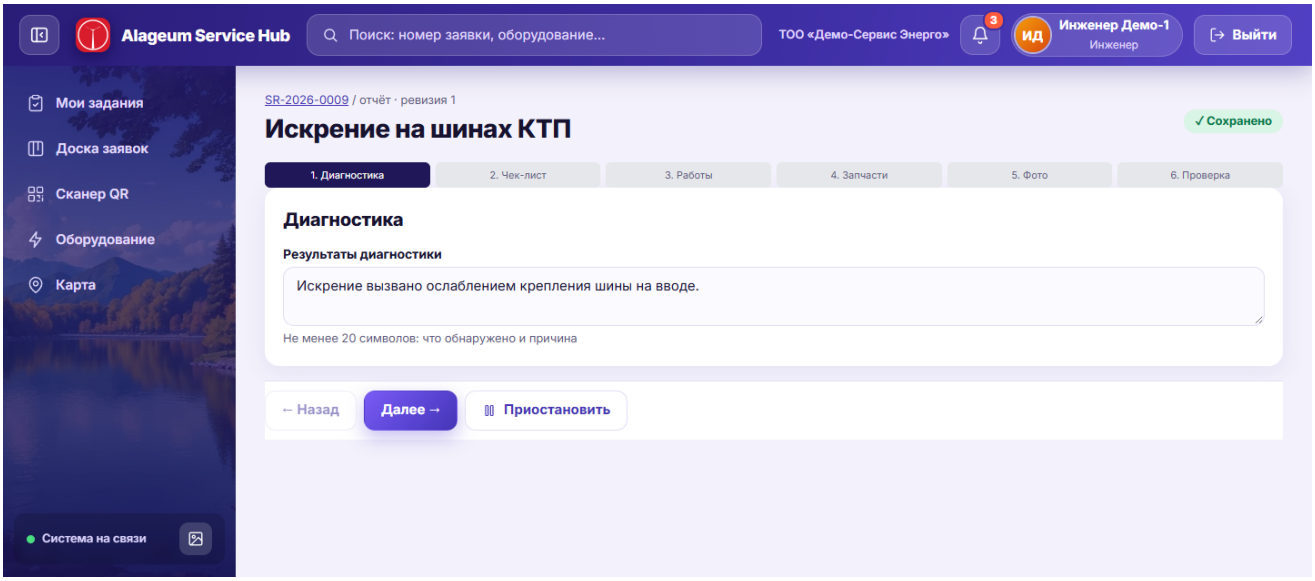


Рисунок 15. Рабочий отчёт инженера по выполняемой заявке

Отчёт содержит диагностику, выполненные работы, итог, рекомендации и чек-лист. Для результата с замечаниями обязательны рекомендации. Поля проверяются на сервере: диагностике и описанию работ требуется не менее 20 символов, итогу — не менее 10.

Завершение возможно после заполнения обязательных пунктов чек-листа. Для значения вне нормы требуется комментарий. Также необходимы фотографии «до» и «после» либо согласованное диспетчером исключение. Список материалов должен быть подтверждён, даже если запчасти не использовались.

Проверка готовности возвращает конкретные причины, мешающие завершению. Эти требования повышают полноту сервисного отчёта, но не подтверждают техническую правильность выполненного ремонта: содержание и качество работ остаются ответственностью профильного специалиста.

Основа описания: backend/app/execution.py completion_check; frontend/src/pages/Engineer.tsx

18 Запчасти и использованные материалы

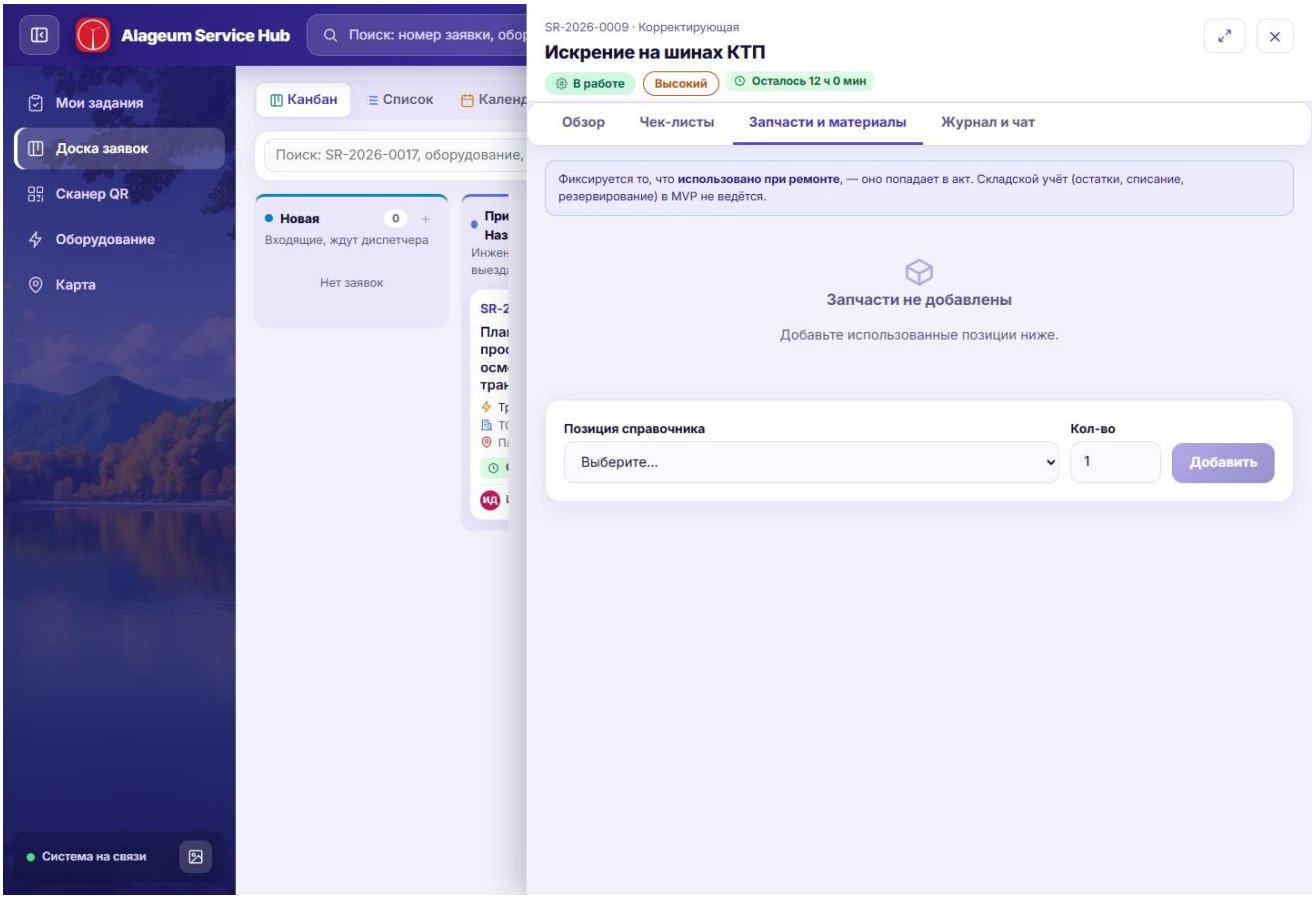


Рисунок 16. Материалы в контексте отчёта по заявке

Справочник запчастей используется для единообразного выбора материалов. В отчёте фиксируются применённые позиции, количество и сведения, необходимые для описания выполненной работы. Данные привязаны к конкретной редакции отчёта и включаются в акт.

Инженер подтверждает список перед завершением. Если материалы не применялись, это также должно быть явно отмечено. Такая проверка уменьшает число отчётов, в которых отсутствие записей невозможно отличить от незаполненной вкладки.

В текущем MVP реализован учёт факта использования. Складские остатки, резервирование, закупки и автоматическое списание не входят в этот механизм. При последующей интеграции со складской системой потребуется определить владельца справочника, единицы измерения и правила обмена.

Основа описания: frontend/src/board/Drawer.tsx PartsTab; backend/app/routers/execution.py

19 Журнал заявки и коммуникация

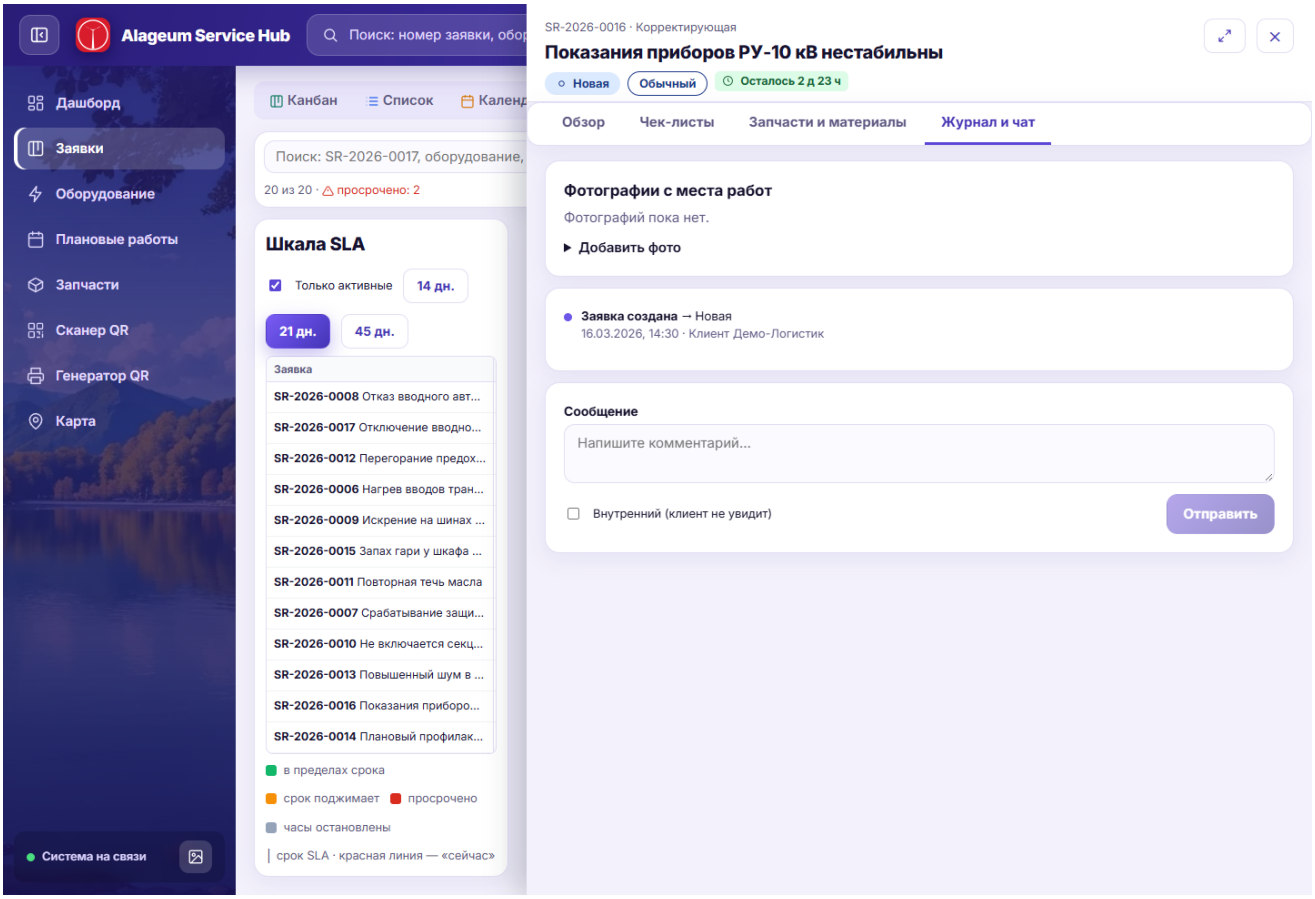


Рисунок 17. История событий и комментарии в боковой панели заявки

Боковая панель позволяет просмотреть обращение, не уходя с доски. Вкладки объединяют обзор, чек-листы, материалы и журнал. В обзоре видны оборудование, SLA, доступные действия и связанная история клиента.

Журнал сохраняет события процесса: создание, принятие, назначение, начало, паузы, завершение и последующие решения. Комментарии дополняют формальные события рабочими пояснениями. Вложения связаны с заявкой и проверяются по той же области доступа.

Разделяются публичные и внутренние записи. Клиент получает предназначенную ему часть истории; внутренние сведения сервисной команды не должны автоматически становиться публичными. Это важно учитывать при написании комментария и выборе видимости фотографии или документа.

Основа описания: frontend/src/board/Drawer.tsx; backend/app/serializers.py events_for; routers/files.py

20 Акт и подтверждение клиента

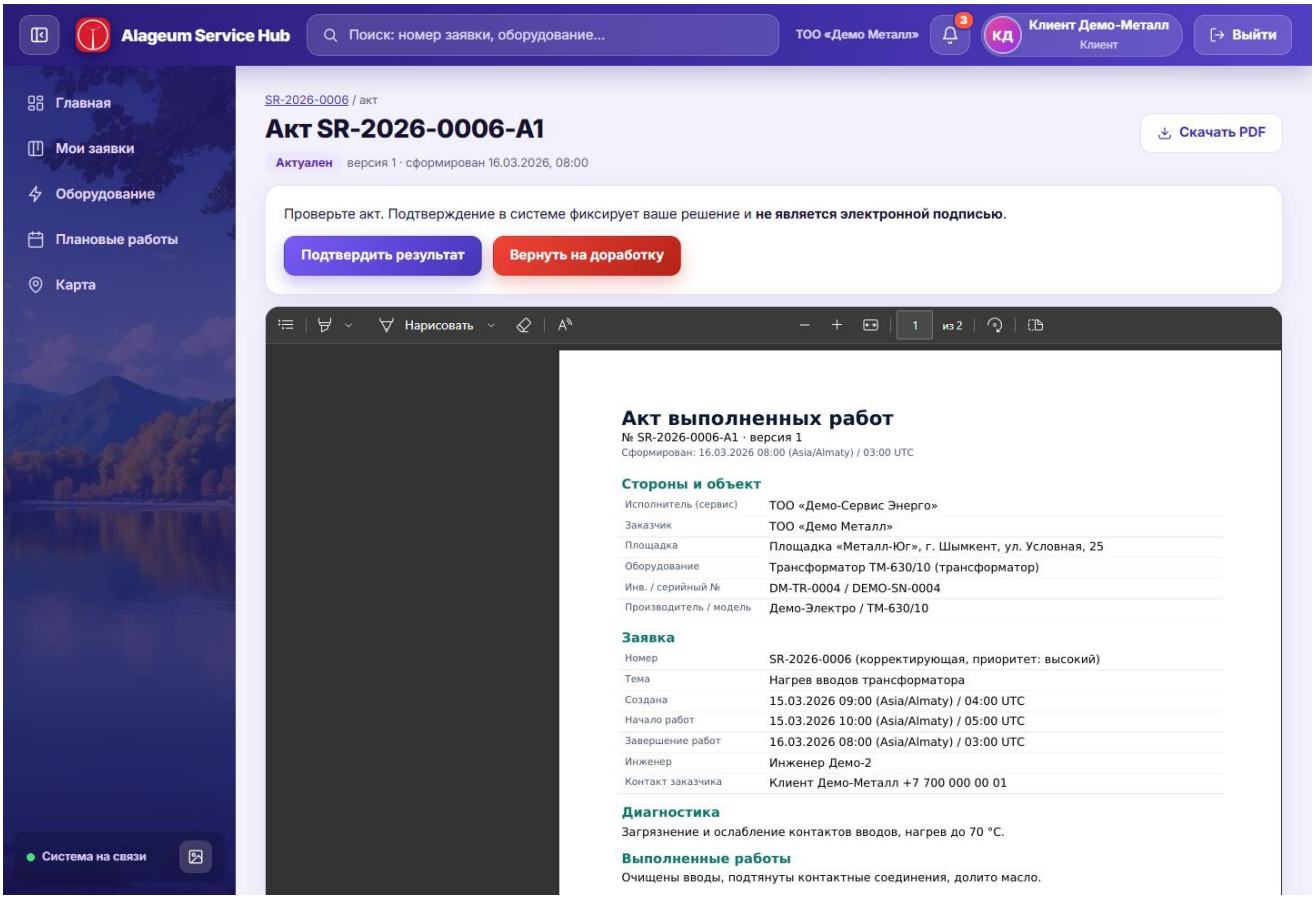


Рисунок 18. Акт выполненных работ по демонстрационной заявке

После успешного завершения работы система формирует акт в PDF на основе отчёта и снимка данных. Акт связан с заявкой и редакцией отчёта; файл имеет размер и контрольную сумму SHA-256. Снимок позволяет сохранить содержание документа при последующих изменениях справочников.

Клиент просматривает результат и подтверждает его либо возвращает на доработку с замечаниями. Подтверждение закрывает заявку. При возврате сохраняются предыдущие документы и создаётся дальнейшая история работы; актуальная версия акта выделяется отдельно.

В системе предусмотрен также PDF-лист подтверждения. Действие клиента в веб-интерфейсе следует описывать как подтверждение в системе. Встроенная квалифицированная электронная подпись и интеграция с ЭЦП в рассматриваемой реализации не заявляются.

Основа описания: backend/app/acts.py; pdf.py; routers/misc.py; workflow.py

21 Уведомления и контроль внимания

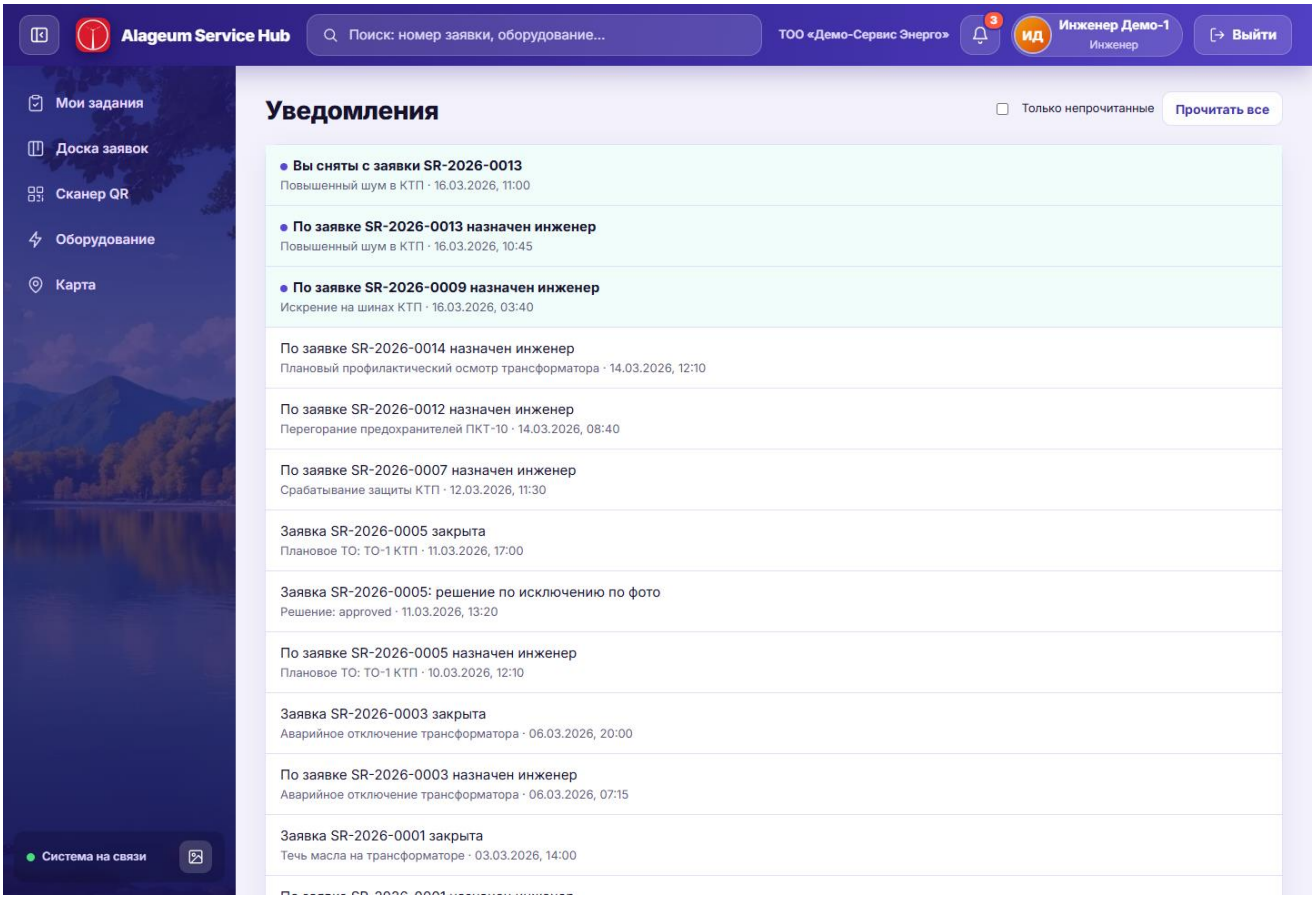


Рисунок 19. Центр уведомлений пользователя

Уведомления сообщают о событиях, требующих внимания: регистрации обращения, назначении, изменении хода работ и решении по результату. Счётчик в навигации показывает непрочитанные записи. Из сообщения можно перейти к связанному обращению.

Worker формирует отдельные напоминания о просрочке SLA, затянувшейся паузе и ожидании подтверждения. Для ожидания подтверждения в коде установлен порог 72 часа. Ключи дедупликации помогают не создавать одинаковое уведомление при каждом запуске фоновой проверки.

Прочтение уведомления не изменяет статус заявки и не заменяет соответствующее действие процесса. В текущем продукте это внутренний центр уведомлений; интеграции с email, SMS и мессенджерами не следует считать включёнными без отдельной реализации.

Основа описания: backend/app/notifications.py; jobs.py; routers/misc.py

22 Пользователи и административный доступ

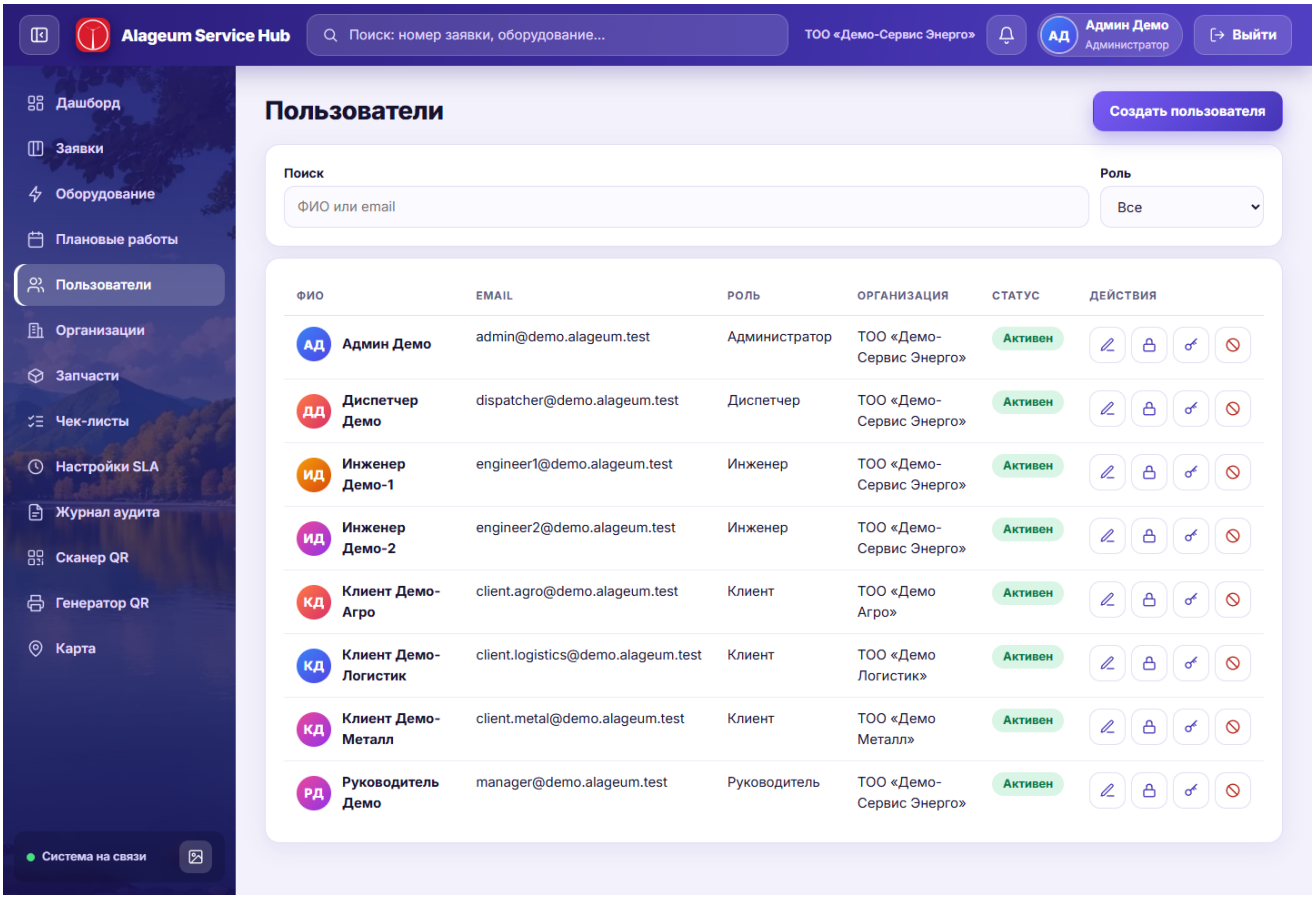


Рисунок 20. Реестр пользователей с ролями и действиями администратора

Администратор создаёт учётные записи, связывает их с организацией, назначает роль и управляет активностью. Для инженеров задаётся вместимость, используемая в оценке загрузки. Руководителю доступен просмотр пользователей в пределах предусмотренных API прав.

При создании или изменении можно задать пароль, сгенерировать его и включить требование смены при следующем входе. Минимальная длина — 12 символов; отклоняются явно простые значения и совпадения с идентификационными данными. Назначение нового пароля отзывает прежние сеансы.

Пароли хранятся в виде хэшей Argon2. Временные значения показываются в предусмотренном интерфейсом сценарии, поэтому их передачу пользователю следует организовать отдельно. Пароли, cookie и секреты окружения не должны включаться в презентационные материалы или инструкции общего доступа.

Основа описания: frontend/src/pages/Admin.tsx; backend/app/routers/users.py; security.py

23 Организации площадки и справочники

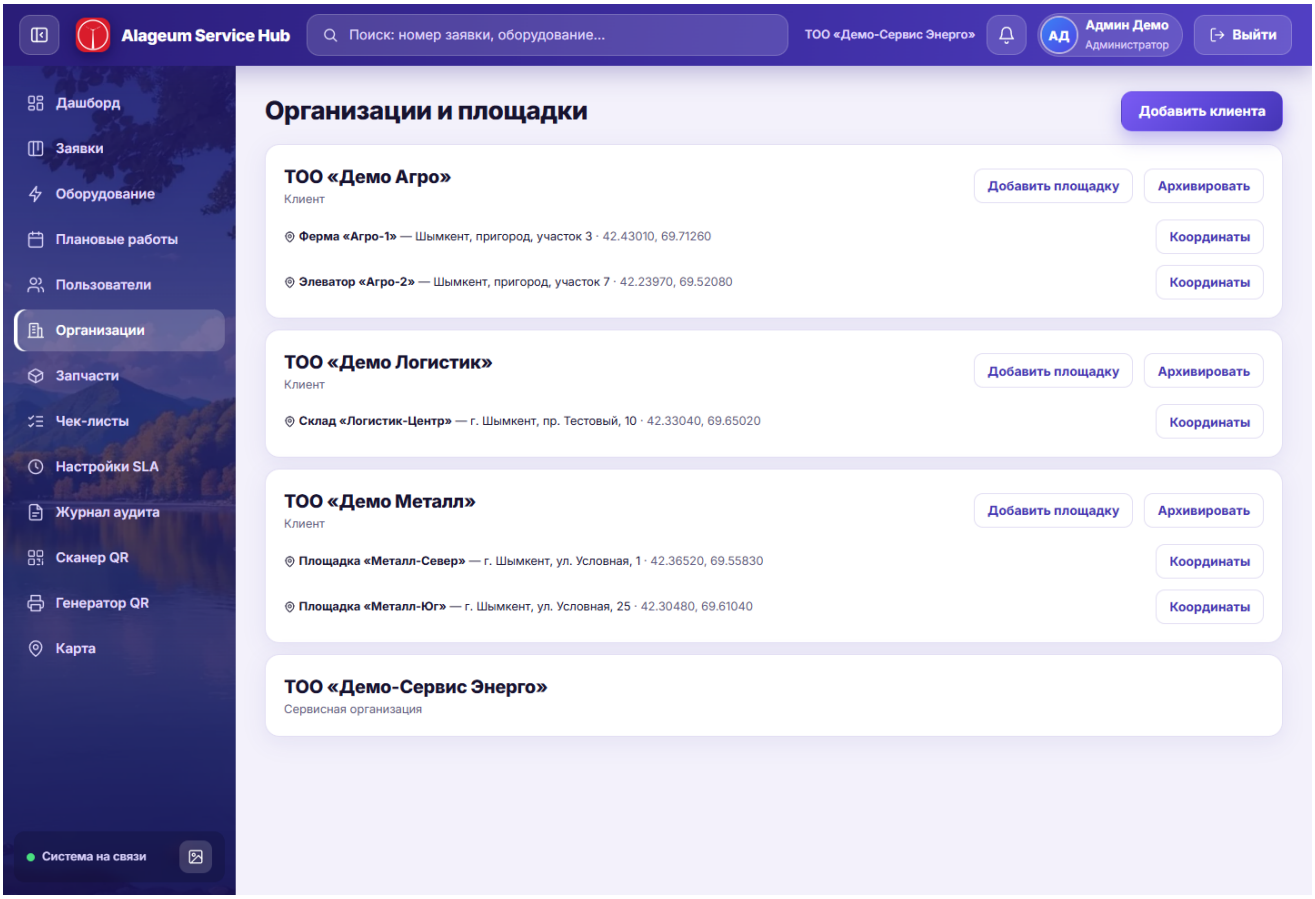


Рисунок 21. Структура организаций и площадок

Организация определяет владельца оборудования и границу клиентских данных. Площадка уточняет физическое размещение: название, адрес, контакты, часовой пояс и координаты. В модели предусмотрена одна сервисная организация и несколько клиентских.

Связи между организацией, площадкой и оборудованием контролируются внешними ключами и ограничениями базы данных. Такой контроль дополняет проверку форм и препятствует появлению логически несогласованных записей.

Перед загрузкой реальных данных следует согласовать правила именования и архивирования. Для эксплуатации особенно важны корректные контакты, часовой пояс площадки и координаты. Переименование организации не должно приводить к созданию дубликата всей истории оборудования.

Основа описания: backend/app/models.py Organization and Site; routers/orgs.py

24 Шаблоны чек листов и настройки SLA

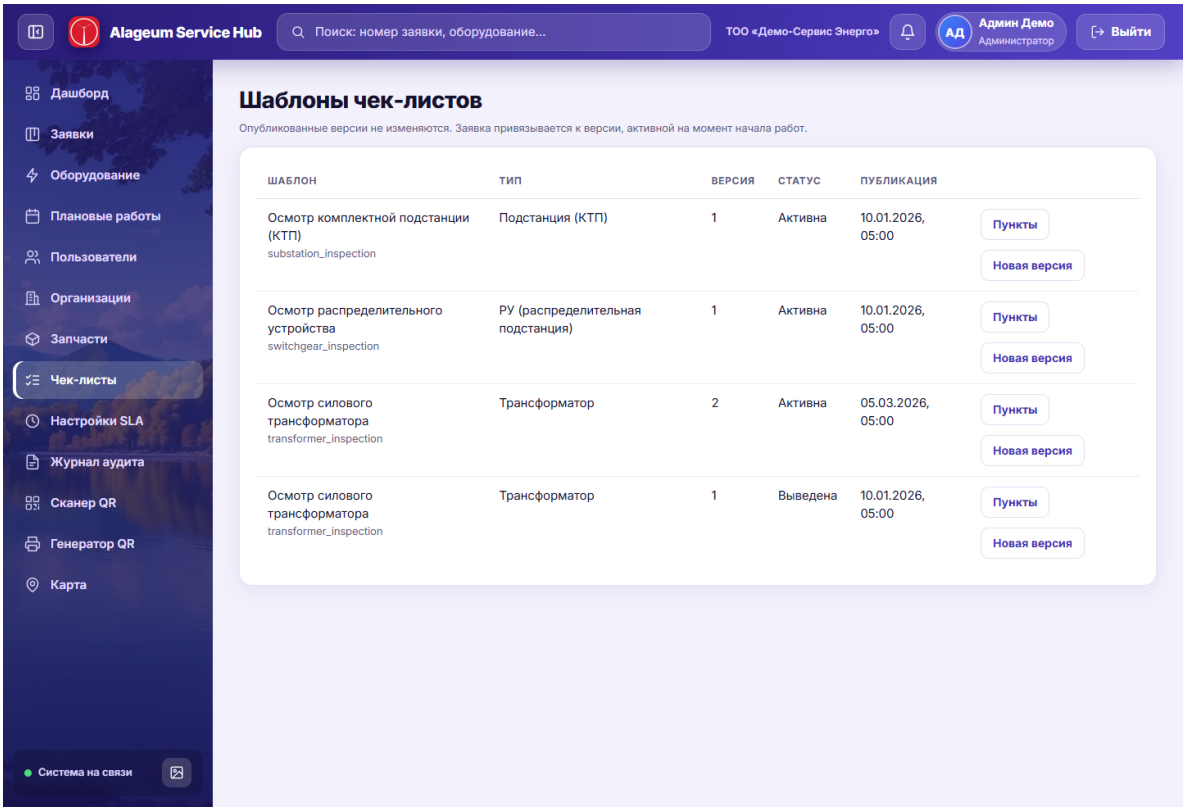


Рисунок 22. Шаблоны проверок для разных типов оборудования

Чек-лист задаёт структуру проверки для типа оборудования: вопросы, типы ответов, единицы измерения и обязательность. Шаблоны имеют версии и состояния черновика, активной и выведенной версии. Отчёт ссылается на конкретный шаблон, что сохраняет смысл ранее собранных ответов.

Политики SLA задаются по приоритету. Значения ниже являются исходными настройками реализации; перед эксплуатацией их следует согласовать как правила сервиса. Длительности считаются в календарном времени с учётом остановки часов выполнения в соответствующих статусах.

Приоритет	Реакция	Выполнение
Критический	60 минут	480 минут
Высокий	240 минут	1440 минут
Обычный	480 минут	4320 минут
Низкий	1440 минут	10080 минут

Основа описания: backend/app/sla.py DEFAULT_POLICY; routers/execution.py; routers/misc.py

25 Мобильный сценарий

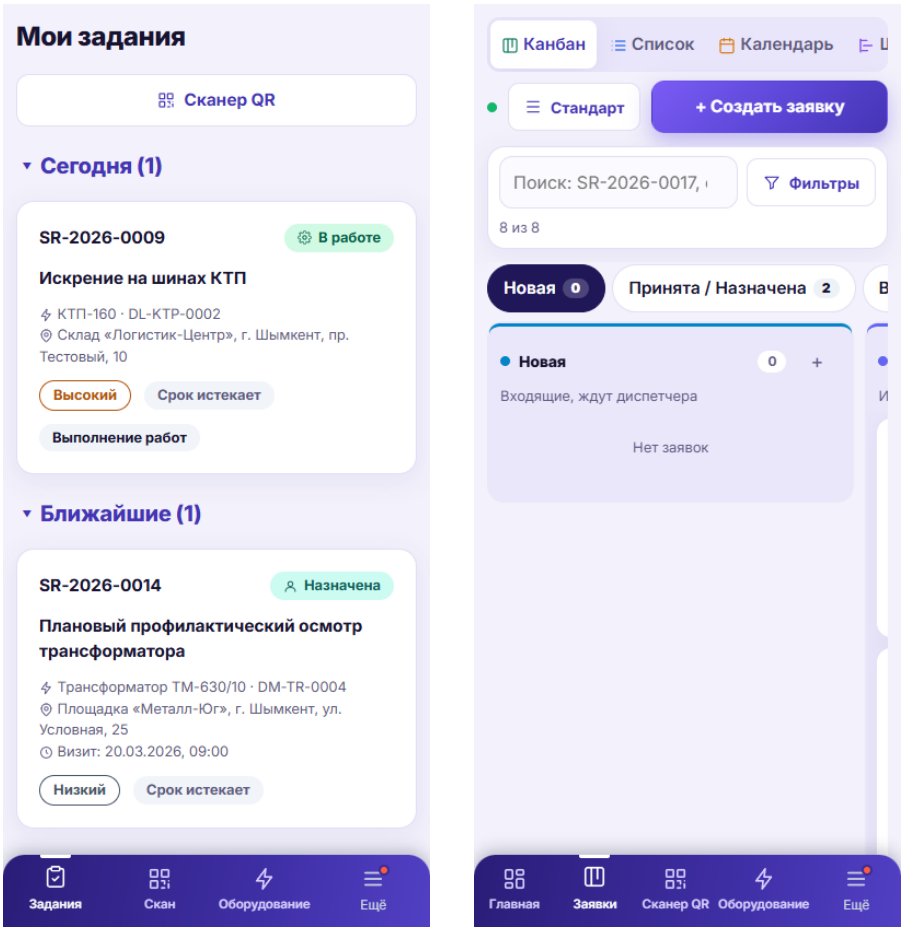


Рисунок 23. Мобильные представления заданий инженера и доски клиента

Адаптивный интерфейс предназначен для работы через браузер телефона. Инженер открывает задания, переходит к объекту, заполняет отчёт и добавляет фотографии. Клиент отслеживает свои обращения и получает доступ к результатам.

Нижняя панель сохраняет основные переходы под рукой; меню «Ещё» содержит дополнительные разделы и профиль. Камера и геолокация требуют соответствующих разрешений. Показанные экраны сняты при ширине 390 пикселей в браузерной эмуляции; это не подтверждение испытаний на всех физических телефонах.

При потере связи интерфейс сообщает об отсутствии соединения. Сохранённые данные могут оставаться видимыми, но полноценная автономная работа с последующей синхронизацией изменений в MVP не заявляется.

Основа описания: frontend/src/App.tsx; frontend/e2e/30-mobile.spec.ts; 50-responsive.spec.ts

26 Архитектура приложения

Приложение построено как веб-платформа с отдельным клиентом, серверным API и фоновым обработчиком. Браузер загружает React-приложение через Nginx. Запросы к API проксируются на FastAPI, который проверяет сессию, права, входные данные и правила предметной области.

Основные записи хранятся в PostgreSQL. Фотографии, вложения и акты размещаются в отдельном файловом томе и выдаются через API с проверкой доступа. Worker использует тот же серверный образ и выполняет периодические операции.

Компонент	Технология	Назначение
Интерфейс	React 19, TypeScript, Vite	Страницы, формы, адаптивная навигация
Состояние запросов	TanStack Query	Загрузка, кэш и обновление данных
Доска и карта	dnd-kit, Leaflet	Перетаскивание и географическое представление
API	FastAPI, Python 3.12	Авторизация, правила процесса, REST
Доступ к БД	SQLAlchemy, Alembic	Модели, транзакции и миграции
База данных	PostgreSQL 16	Сущности, ограничения и события
Документы	ReportLab	Акты, паспорта и этикетки PDF
Публикация	Nginx, Docker Compose	Статика, прокси и запуск сервисов

Поток изменения заявки: браузер отправляет команду → API проверяет её → транзакция обновляет состояние и события → после фиксации приходит уведомление SSE → интерфейс перечитывает данные по REST. Таким образом, обновление экрана использует обычную проверку прав.

Версии зависимостей закреплены в package-lock.json и Python lock-файлах. Внешний вид строится на CSS-токенах и компонентах проекта; Tailwind CSS в текущей реализации не используется.

Основа описания: compose.yaml; frontend/package.json; backend/requirements.lock; app/realtime.py

27 Модель данных

Центральная сущность — заявка Ticket. Она связывает оборудование, клиента, площадку, текущего инженера, приоритет и состояние. События и назначения вынесены отдельно, поэтому текущее состояние не заменяет историю.

У большинства сущностей идентификаторы UUID. Для последовательных журналов используются числовые идентификаторы. Внешние ключи, уникальные индексы и CHECK-ограничения поддерживают согласованность данных на уровне PostgreSQL.

Группа	Основные таблицы	Связи и назначение
Участники	organizations, sites, users	Организация владеет площадками; пользователь имеет роль
Активы	equipment, maintenance_plans	Оборудование относится к площадке; планы — к оборудованию
Обращения	tickets, assignments	Заявка и история назначений исполнителей
История	ticket_events, comments	Формальные события и рабочие сообщения
Исполнение	work_reports, checklist_templates	Редакции отчётов и версии шаблонов
Материалы	spare_parts, part_usages	Справочник и использование в отчётах
Файлы	attachments, acts	Метаданные вложений, PDF и снимки актов
Доступ	sessions, audit_log	Сеансы и журнал административных событий
Служебные	notifications, idempotency_keys, job_runs	Уведомления, защита повторов, фоновые запуски

Поля version защищают изменяемые записи от потери конкурентных изменений. При несовпадении версии сервер возвращает конфликт, а пользователь должен обновить данные перед повторением действия.

JSONB применяется там, где структура меняется по шаблону или типу: паспортные данные, ответы чек-листа, полезная нагрузка событий и снимок акта. Основные связи и ограничения при этом остаются реляционными.

Основа описания: backend/app/models.py; backend/alembic/versions/

28 Контракт API

API имеет префикс /api/v1 и использует JSON для основных операций. Списки возвращают items и сведения о пагинации. Размер страницы ограничен 100 записями; фильтры и сортировка проверяются сервером. Файлы и PDF выдаются отдельными маршрутами.

Авторизация основана на cookie сессии. Для изменяющих запросов используется CSRF-защита; для предусмотренных POST-команд — Idempotency-Key. Поле version в теле операции позволяет обнаружить изменение записи другим пользователем.

Маршрут после /api/v1	Метод	Назначение
/auth/login и /auth/me	POST / GET	Вход и текущий пользователь
/equipment	GET / POST	Реестр и создание оборудования
/tickets	GET / POST	Поиск и регистрация заявок
/tickets/{id}/assign	POST	Назначение инженера
/tickets/{id}/report	GET / PUT	Чтение и сохранение отчёта
/tickets/{id}/report/completion-check	GET	Проверка готовности к завершению
/acts/{id}/pdf	GET	Получение акта
/analytics/summary	GET	Сводная аналитика
/map/objects	GET	Данные объектов для карты
/events/stream	GET	Поток уведомлений SSE

Основные ответы ошибок: 401 — требуется вход, 403 — действие запрещено, 404 — объект отсутствует или не виден, 409 — конфликт версии или состояния, 422 — неверные данные. Идентификатор X-Request-Id помогает сопоставить ответ и серверный журнал.

Повтор с тем же ключом идемпотентности и тем же запросом возвращает сохранённый результат. Использование ключа с другим содержимым отклоняется. В текущей реализации запись ключа хранится 24 часа.

Основа описания: backend/app/routers/; idempotency.py; main.py

29 Защита доступа и файлов

Защита строится на серверных проверках, а не только на скрытии кнопок. Единые функции области видимости ограничивают заявки, оборудование, площадки и связанные файлы. Для инженера дополнительно проверяется текущее назначение при выполнении рабочих действий.

Пароль хэшируется Argon2. Cookie сессии имеет HttpOnly и SameSite=Lax; флаг Secure определяется окружением. Сервер хранит хэш токена сессии, а CSRF-токен вычисляет через HMAC. Предусмотрена временная блокировка после серии неуспешных входов.

Механизм	Реализованное поведение
Сеанс	Ограничение бездействия и абсолютного срока; отзыв при смене пароля
Конкурентность	Проверка version и блокировка записи в транзакции действия
Повторы	Идемпотентность предусмотренных команд по UUID ключа
Изображения	JPEG, PNG, WebP; декодирование, перекодирование и удаление EXIF
PDF	Проверка поддерживаемого формата и размера
Размеры по умолчанию	Фото до 10 МБ, документы до 20 МБ, до 40 вложений заявки
Хранилище	Приватный том, метаданные и контрольные суммы SHA-256
Аудит	События входа, доступа и административных действий

Проверка PDF в текущем коде не равнозначна антивирусному анализу. Перед внешней эксплуатацией следует отдельно оценить обработку недоверенных файлов, защиту канала, управление секретами и полноту журнала.

В режиме pilot конфигурация требует Secure-cookie, запрещает demo-seed и отклоняет стандартный небезопасный секрет. Эти проверки полезны, но не заменяют самостоятельную приёмку производственного окружения.

Основа описания: backend/app/security.py; config.py; routers/files.py; storage.py

30 Расчёт SLA и аналитики

Система различает срок реакции и срок выполнения. Реакция измеряется от создания до первого реагирования. Время выполнения накапливается в состояниях new, accepted, assigned и in_progress; на паузе и при ожидании подтверждения часы выполнения остановлены.

При переходе сохраняется уже израсходованное время. После возобновления расчёт продолжается из накопленного значения. Изменение приоритета меняет лимит и пересчитывает оставшийся срок, сохраняя уже потраченное время.

Показатель	Правило расчёта
Активные заявки	Новые, принятые, назначенные, в работе, на паузе и ожидающие подтверждения
Просрочка реакции	Новая заявка с превышенным сроком первой реакции
Просрочка выполнения	Накопленное время выше лимита; ожидание подтверждения исключено
Средняя реакция	Среднее от создания до первого реагирования по подходящей выборке
Среднее выполнение	Среднее накопленное время завершённых работ, ожидающих подтверждения или закрытых
Выполнено в срок	Доля завершений с накопленным временем не выше лимита
Повторные неисправности	Новая корректирующая заявка в течение 30 дней после закрытия другой по тому же оборудованию
Загрузка инженера	Число assigned, in_progress, paused относительно capacity

Времена хранятся с часовым поясом; бизнес-логика работает с UTC. Для пользовательских дат применяется контекст площадки или интерфейса. Демонстрационные бизнес-часы отделены от времени безопасности сеансов.

Текущий расчёт сводной аналитики выполняется в Python по видимым заявкам. При существенном росте объёма следует измерить время ответа и при необходимости переносить агрегации ближе к базе данных. Целевые показатели производительности не следует считать достигнутыми без замеров.

Основа описания: backend/app/sla.py; metrics.py; clock.py

31 Обновления и фоновые задачи

Для живого обновления используется Server-Sent Events. PostgreSQL доставляет уведомления через LISTEN/NOTIFY, а сервер передаёт краткое событие подписанным браузерам. Изменения становятся доступны после фиксации транзакции.

Получив событие, интерфейс обновляет соответствующие REST-запросы. Данные снова проходят стандартную проверку доступа. Поток передаёт идентификатор заявки, номер, статус и тип события; он не заменяет основной API чтения.

Задача	Назначение	Защита от повторов
Создание плановых заявок	Перевод наступивших планов в рабочую очередь	Уникальная пара план и плановая дата
Контроль SLA	Сообщения о нарушении реакции и выполнения	Ключ уведомления по заявке и событию
Напоминания	Просроченная пауза и ожидание подтверждения	Дедупликация уведомлений
Очистка	Устаревшие служебные записи и файловые сироты	Проверка сроков и ссылок
Проверка хранилища	Поиск отсутствующих файлов и несовпадений хэшей	Сопоставление БД и тома files

Worker удерживает advisory-lock PostgreSQL, чтобы активным планировщиком был один экземпляр. Heartbeat используется для проверки жизнеспособности. Основной цикл выполняется примерно раз в минуту; периодическая очистка включается раз в 60 циклов.

Сбой фонового задания записывается в журнал и откатывает его транзакцию. Для эксплуатации нужно наблюдать не только доступность веб-страницы, но и работоспособность worker: без него новые плановые заявки и напоминания могут задерживаться.

Основа описания: backend/app/realtime.py; routers/events.py; worker.py; jobs.py

32 Развёртывание и конфигурация

Базовый запуск организован через Docker Compose. Сервисы запускаются с зависимостями: база данных, миграции, API, worker и Nginx. Проверки состояния позволяют дождаться готовности перед выдачей пользовательского интерфейса.

HTTP опубликован на порту 8100, HTTPS — на 8443. Сертификаты монтируются из каталога certs. Для телефона в локальной сети нужен адрес компьютера, а для устойчивой эксплуатации предпочтителен постоянный адрес сервиса с корректным сертификатом.

Параметр	Назначение
APP_ENV	Режим dev, test, demo или pilot
APP_PUBLIC_URL	Адрес для QR и ссылок; допускается auto
APP_CLOCK_MODE	Системные, фиксированные или сдвинутые бизнес-часы
COOKIE_SECURE	Передача cookie только по защищённому каналу
SESSION_IDLE_TTL_MINUTES	Предел бездействия сеанса
SESSION_ABS_TTL_MINUTES	Абсолютный предел сеанса
WORKER_ENABLED	Включение фоновой обработки
ALLOW_DEMO_SEED	Разрешение загрузки демонстрационных данных

Типовая последовательность на подготовленном хосте: создать конфигурацию и секреты, собрать образы командой docker compose build, затем выполнить docker compose up -d --wait. Демо-наполнение запускается отдельно и только в предназначенном для него окружении.

Файлы .env и приватные ключи не включаются в общий документ. Изменение конфигурации должно сопровождаться проверкой входа, API, worker, доступа к файлам и QR-ссылок с целевого устройства.

Основа описания: compose.yaml; .env.example; backend/app/config.py; scripts/make_cert.sh

33 Резервное копирование и сопровождение

Рабочее состояние состоит из базы PostgreSQL и файлового тома. Копирование только базы недостаточно: в ней находятся ссылки на фотографии и PDF. В проекте предусмотрены scripts/backup.sh и scripts/restore.sh для совместной работы с этими частями.

Резервная копия содержит db.dump, files.tar.gz и manifest.json. В манифест записываются время, версия приложения, ревизия миграций, число заявок и контрольные суммы. Дамп выполняется раньше архива файлов; при активных изменениях нужно отдельно обеспечить согласованность выбранного момента.

Операция	Команда или результат
Состояние сервисов	docker compose ps
Последние журналы	docker compose logs --tail=200 api worker web
Создание копии	bash scripts/backup.sh
Проверка файлов	docker compose run --rm api python -m app.cli verify-storage
Восстановление	bash scripts/restore.sh backups/<метка>
Готовность API	GET /api/v1/health/ready

Восстановление заменяет текущие данные базы и файлов. Его следует сначала испытать на отдельном стенде с выбранной копией. После восстановления нужно проверить вход, наличие заявок, открытие вложений и актов, а затем выполнить verify-storage.

Расписание копирования, внешнее хранение, срок хранения и цели восстановления должны быть определены перед пилотом. При проверке хэшей нельзя полагаться только на завершение скрипта: текущий restore.sh выводит предупреждение при невозможности проверки и может продолжить работу.

Основа описания: scripts/backup.sh; scripts/restore.sh; backend/app/jobs.py verify_storage

34 Приёмка и дальнейшее развитие

Демонстрацию удобно проводить последовательно по ролям: клиент регистрирует проблему, диспетчер назначает инженера, инженер оформляет результат, клиент подтверждает акт, руководитель проверяет показатели. Такой проход связывает презентацию интерфейса с проверкой процесса.

В репозитории имеются тесты pytest, Vitest и Playwright, включая сценарии ролей, заявок, мобильного интерфейса, QR, карты и сеансов. Исторические отчёты о прогонах относятся к предыдущим состояниям проекта; при выпуске новой версии требуется отдельный актуальный прогон.

Проверка перед пилотом	Ожидаемый результат
Изоляция клиентов	Пользователь не читает чужие объекты и вложения
Полный цикл заявки	Сохраняются роли, события, отчёт, акт и решение
Конфликт изменений	Устаревшая версия не перезаписывает новые данные
SLA и паузы	Расчёт соответствует согласованным правилам
Мобильные устройства	Камера, фото, формы и геолокация проверены на целевых телефонах
Резервное восстановление	База и файлы восстановлены и сверены
Производительность	Время ответа измерено на ожидаемом объёме
Безопасность окружения	Проверены TLS, секреты, права и обработка файлов

Границы текущего MVP: отсутствуют полноценный склад, заявленная интеграция с 1С или ERP, ЭЦП и гарантированная автономная работа. Производственная нагрузка, реальные устройства и готовность конкретного размещения требуют самостоятельной приёмки.

Первоочередное развитие — согласовать регламенты и справочники, провести ограниченный пилот, измерить нагрузку и качество данных, затем выбирать интеграции. Основа документа: исходный код ревизии 8065c14, конфигурация проекта и экраны локального demo-стенда.

Основа описания: docs/12-implementation-status.md; backend/tests/; frontend/e2e/; README.md